



SPOŁECZNA AKADEMIA NAUK

www.san.edu.pl

Kierunek studiów: **Informatyka**

Nazwisko i Imię: Roman Piotr

Numer albumu: 122217

Grupa: IV

nr grupy: 4

Studia Inż. Zaoczne

Systemy Szkieletowe

Sprawozdanie

z projektu

**Aplikacja kierowcy – system zarządzania
dokumentami przewozowymi**

Prowadzący zajęcia: **mgr inż. Krystian Gumiński**

Łódź 2025/2026

Spis treści

1	Wstęp	3
2	Cel projektu	4
2.1	Wymagania funkcjonalne	4
2.2	Wymagania niefunkcjonalne	4
3	Przegląd istniejących rozwiązań	5
3.1	Tłó: elektroniczny list przewozowy	5
3.2	TransFollow	5
3.3	Platforma Trans.eu	6
3.4	Adobe Scan	6
3.5	Porównanie	6
4	Opis architektury	8
4.1	Wybór technologii	9
4.2	Struktura repozytorium	9
5	Opis implementacji	11
5.1	Frontend	11
5.1.1	Struktura aplikacji	11
5.1.2	Logowanie i sesja	11
5.1.3	Formularze	12
5.1.4	Układ dla urządzeń mobilnych	12
5.1.5	Dokumenty i profil	12
5.1.6	Zdjęcie dokumentu aparatem	13
5.1.7	Aplikacja instalowalna (PWA)	15
5.1.8	Panel biura	16
5.1.9	Wygląd aplikacji	17
5.2	Backend	17
5.2.1	Warstwy aplikacji	17
5.2.2	Walidacja konfiguracji	20
5.2.3	Obsługa błędów	20
5.2.4	Logowanie zdarzeń	20
5.2.5	Punkty kontroli stanu	21
5.2.6	Rejestracja i logowanie	21
5.2.7	Ochrona przed zgadywaniem haseł	23
5.2.8	Profil kierowcy	23
5.2.9	Dokumentacja API	25
5.2.10	Dokumenty	25

5.2.11	Role i panel biura	28
5.3	Baza danych	29
5.3.1	Migracje	29
5.3.2	Model danych	29
5.4	Docker	31
5.4.1	Wieloetapowe obrazy	31
5.4.2	Serwer nginx	32
5.4.3	Kolejność uruchamiania i odporność na awarie	33
5.4.4	Sieci i wolumeny	33
5.4.5	Konfiguracja i sekrety	33
5.4.6	Środowisko testowe	34
5.4.7	Napotkany problem: obrazy MinIO	34
5.5	Testy	34
5.5.1	Strategia	34
5.5.2	Środowisko testowe	35
5.5.3	Zakres testów	35
5.5.4	Błędy wykryte przez testy	35
5.5.5	Stabilność testów w przeglądarce	37
5.5.6	Testy obrazów Docker	38
5.5.7	Test na prawdziwym telefonie	38
5.5.8	Pokrycie kodu	38
5.5.9	Wyniki uruchomienia testów	40
5.5.10	Ciągła integracja	43
5.6	Monitoring	43
5.6.1	Metryki	44
5.6.2	Dashboardy	45
5.6.3	Watchdog procesu	45
5.6.4	Watchdog kontenerów	47
5.6.5	Sondy z zewnątrz	47
5.6.6	Alerty	47
5.6.7	Logi	49
5.6.8	Test watchdogów (chaos test)	49
6	Podsumowanie i wnioski	50
6.1	Realizacja celu	50
6.2	Napotkane problemy i wnioski	50
6.3	Ograniczenia	51
6.4	Możliwości rozwoju	51
	Bibliografia	55

1 Wstęp

Kierowca w transporcie drogowym przez cały czas pracy korzysta z dokumentów przewozowych. Najważniejszym z nich jest międzynarodowy list przewozowy CMR, którego rolę i zawartość określa Konwencja o umowie międzynarodowego przewozu drogowego towarów [1]. W przewozach krajowych i przy odbiorze towaru z magazynu dochodzą do tego dokumenty wydania zewnętrznego (WZ), faktury i inne potwierdzenia. Większość z nich nadal ma formę papierową: kierowca otrzymuje je przy załadunku, zbiera pieczętki i podpisy przy rozładunku, a potem dostarcza oryginały do biura firmy przewozowej.

Papierowy obieg dokumentów powoduje typowe problemy. Dokumenty giną lub niszczą się w kabinie pojazdu, biuro poznaje ich treść dopiero po powrocie kierowcy, a rozliczenie zlecenia i wystawienie faktury się opóźnia. Kierowca ma jednak przy sobie telefon z aparatem, więc może dokument sfotografować i przesłać go zaraz po otrzymaniu.

Niniejsza praca opisuje projekt i implementację *Aplikacji kierowcy*: responsywnej aplikacji webowej (PWA), w której kierowca fotografuje dokument telefonem, kadruje i kompresuje zdjęcie, a następnie przechowuje je w zaszyfrowanej postaci, z podziałem na typy, statusy i wersje. Rozdział 2 określa cel projektu i wymagania, rozdział 3 przedstawia istniejące rozwiązania, a rozdziały 4 i 5 opisują architekturę i implementację systemu. Pracę zamyka podsumowanie z wnioskami (rozdział 6).

2 Cel projektu

Celem projektu jest aplikacja, dzięki której kierowca ma dokumenty przewozowe w telefonie, a nie tylko w papierowej teczce. Aplikacja ma skrócić czas od otrzymania dokumentu do jego dostępności w formie cyfrowej, zmniejszyć ryzyko zgubienia dokumentu i zapewnić jego poufność.

2.1 Wymagania funkcjonalne

1. Rejestracja konta (formularz z walidacją danych) i logowanie loginem i hasłem; uwierzytelnianie żądań tokenem JWT.
2. Profil kierowcy, do którego przypisane są jego dokumenty.
3. Zarządzanie dokumentami: typ dokumentu (np. CMR, WZ), status (np. roboczy, przesłany, zaakceptowany), tytuł i numer, wyszukiwanie i filtrowanie.
4. Zdjęcie dokumentu aparatem urządzenia mobilnego (interfejs `getUserMedia`), podgląd i kadrowanie przed zapisaniem oraz kompresja zdjęcia przed wysłaniem na serwer.
5. Wersjonowanie dokumentów: każda nowa wersja pliku jest zachowywana, a zmiany metadanych i statusu trafiają do historii zmian.
6. Panel biura (dyspozytora) w przeglądarce na komputerze: przegląd dokumentów wszystkich kierowców, akceptacja albo odrzucenie przesłanego dokumentu z podaniem powodu. Kierowca widzi decyzję i powód na telefonie i może dokument poprawić. O akceptacji decyduje wyłącznie biuro.

2.2 Wymagania niefunkcjonalne

1. **Bezpieczeństwo:** pliki dokumentów są przechowywane w postaci zaszyfrowanej, hasła jako skróty kryptograficzne, a użytkownik ma dostęp wyłącznie do własnych dokumentów.
2. **Obsługa na urządzeniach mobilnych:** interfejs responsywny, dostosowany do obsługi dotykowej, możliwy do zainstalowania jako PWA.
3. **Łatwe wdrożenie:** cały system uruchamiany jednym poleceniem przez Docker Compose.
4. **Niezawodność i obserwowalność:** monitorowanie stanu zdrowia usług, metryki, alerty i automatyczne restarty po awarii.
5. **Jakość:** testy funkcjonalne każdej funkcji systemu, uzupełnione testami jednostkowymi.

3 Przegląd istniejących rozwiązań

Na rynku są dwie grupy narzędzi bliskich tematowi projektu: platformy transportowe z obiegiem dokumentów elektronicznych oraz ogólne aplikacje do skanowania dokumentów telefonem. Z każdej grupy wybrano rozwiązanie reprezentatywne i porównano je według tych samych kryteriów: przeznaczenie, platforma, obsługa elektronicznego listu przewozowego (e-CMR), zdjęcia papierowych dokumentów, organizacja dokumentów (typy, statusy, wersje), przechowywanie i koszt. Informacje pochodzą z oficjalnych stron producentów i aktów prawnych, stan na wrzesień 2026 r.

3.1 Tło: elektroniczny list przewozowy

Elektroniczny list przewozowy wprowadza Protokół dodatkowy do Konwencji CMR, podpisany w Genewie 20 lutego 2008 r. i obowiązujący od 5 czerwca 2011 r. [2]. Polska przystąpiła do niego w 2019 r. Na poziomie Unii Europejskiej rozporządzenie (UE) 2020/1056 w sprawie elektronicznych informacji dotyczących przewozu towarów (eFTI) zobowiązuje organy państw członkowskich do przyjmowania informacji udostępnionych elektronicznie przez certyfikowane platformy eFTI. Nie nakłada jednak na przewoźników obowiązku rezygnacji z papieru, a wybór formy elektronicznej pozostaje dobrowolny [3]. W najbliższych latach papierowe dokumenty pozostaną więc w użyciu obok elektronicznych, a kierowca nadal musi je przechowywać i przekazywać do biura.

3.2 TransFollow

TransFollow to platforma e-CMR dla firm transportowych [4]. Składa się z interfejsu API do integracji z systemami ERP, WMS i TMS, portalu webowego dla biura, aplikacji mobilnej *TransFollow Drive* dla kierowców (Android i iOS) oraz usługi, w której kierowca komunikuje się przez komunikatory bez instalowania aplikacji. Producent deklaruje zgodność ze standardami UN/CEFACT, RODO i ISO 27001 oraz gotowość do eFTI. W aplikacji *Drive* kierowca rejestruje etapy przewozu, udostępnia lokalizację, zgłasza nieprawidłowości i obsługuje dokumenty elektroniczne: e-CMR, potwierdzenie dostawy, listy pakowe. Dokumenty podpisuje podpisem elektronicznym, a do zlecenia może dołączyć zdjęcia, które trafiają do archiwum w portalu [4].

TransFollow jest rozwiązaniem B2B zbudowanym wokół zlecenia transportowego i współpracy nadawcy, przewoźnika i odbiorcy. Obsługuje dokumenty tworzone elektronicznie, a zdjęcia są jedynie załącznikami. Na stronie producenta nie znaleziono funkcji kadrowania ani rozpoznawania papierowych dokumentów, a wśród typów dokumentów nie ma krajowego dokumentu WZ. Cennik nie jest publiczny.

3.3 Platforma Trans.eu

Trans.eu to giełda transportowa (platforma B2B do handlu przewozami) udostępniana na podstawie odpłatnej umowy i cennika [5]. Dla kierowców przeznaczona jest aplikacja *Loads4Driver*, służąca do komunikacji z kierowcą i przekazywania sygnału monitorowania pojazdu. Wymaga ona konta powiązanego z rolą „Kierowca” w koncie firmy i może być dodatkowo płatna [5]. Dokumenty są związane ze zleceniami transportowymi. Grupa Trans.eu rozwija moduł e-CMR: w 2025 r. opisywała automatyczne generowanie e-CMR powiązanego ze zleceniem, w wersji na komputer oraz Android i iOS, testowane przez wybranych klientów [6]. Dostęp do platformy jest płatny (abonament firmy) [5].

Z perspektywy kierowcy Trans.eu jest narzędziem firmy do pozyskiwania i realizacji zleceń, a nie osobistym archiwum dokumentów. W opisie aplikacji kierowcy nie znaleziono fotografowania i kadrowania dokumentów ani statusów i historii wersji dokumentu.

3.4 Adobe Scan

Najprostszym sposobem digitalizacji papierowego dokumentu jest ogólna aplikacja do skanowania telefonem. Przeglądowi poddano jedną z najpopularniejszych, Adobe Scan [7]. Jest bezpłatny na Android i iOS, automatycznie wykrywa krawędzie dokumentu, koryguje perspektywę, usuwa odblaski i cienie, a tekst rozpoznaje funkcją OCR. Wynik zapisuje jako PDF z możliwością przeszukiwania albo jako JPEG, w folderach w chmurze Adobe Document Cloud (szyfrowanie w czasie przesyłania i przechowywania). Część funkcji wymaga płatnej subskrypcji Acrobat.

Pod względem jakości zdjęcia Adobe Scan przewyższa proste kadrowanie zastosowane w projekcie. Jest to jednak skaner ogólnego przeznaczenia: nie zna typów dokumentów przewozowych, nie ma statusów obiegu dokumentu, historii wersji ani powiązania z profilem kierowcy i nie obsługuje e-CMR. Pliki trafiają do chmury dostawcy, nad którą firma nie ma pełnej kontroli.

3.5 Porównanie

Tabela 1 zestawia omówione rozwiązania z projektowaną aplikacją.

Platformy transportowe rozwiązują obieg dokumentów elektronicznych między firmami, ale nie pomagają kierowcy w uporządkowaniu dokumentów papierowych, które wciąż otrzymuje. Skanery dobrze digitalizują papier, ale nie znają specyfiki dokumentów przewozowych. Projektowana aplikacja wypełnia lukę między tymi grupami. Łączy zdjęcie dokumentu telefonem z typami i statusami dokumentów przewozowych, historią wersji, oceną dokumentu przez biuro i szyfrowanym przechowywaniem na serwerze, który firma może uruchomić u siebie jednym poleceniem.

Tabela 1: Porównanie istniejących rozwiązań z Aplikacją kierowcy

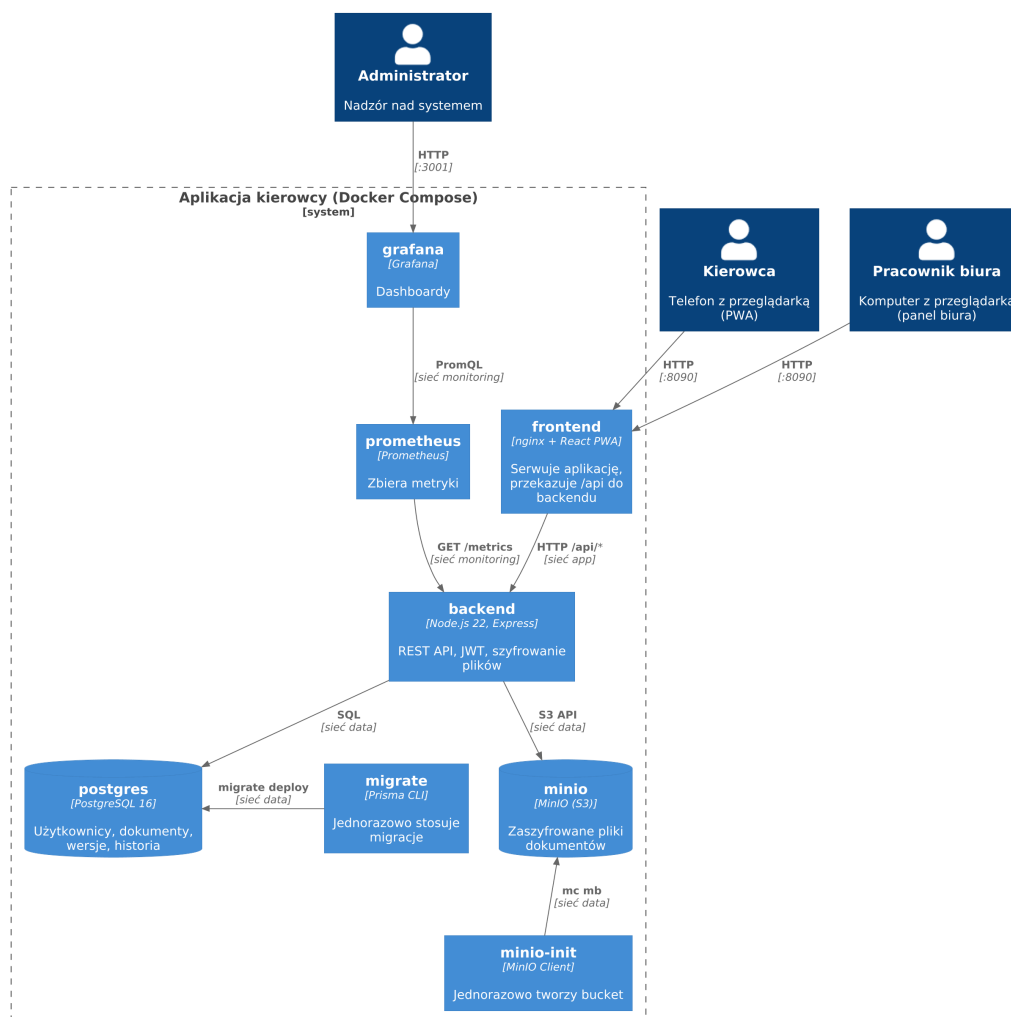
Kryterium	TransFollow	Trans.eu	Adobe Scan	Aplikacja kierowcy
Przeznaczenie	e-CMR i obieg dokumentów między firmami	giełda transportowa i zlecenia	skanowanie dowolnych dokumentów	dokumenty przewożone kierowcy i ich ocena przez biuro
Platforma	Android, iOS, portal WWW, API	WWW, aplikacje Android i iOS	Android, iOS	PWA (kierowca), przeglądarka na komputerze (biuro)
e-CMR	tak (główna funkcja)	w fazie wdrażania	nie	nie (możliwy kierunek rozwoju)
Zdjęcie papierowego dokumentu	zdjęcia jako załączniki	brak informacji	tak: krawędzie, perspektywa, OCR	tak: kadrowanie, obrót, kompresja
Typy i statusy dokumentów	dokumenty zlecenia (e-CMR, POD)	dokumenty zlecenia	brak (foldery)	typy (CMR, WZ, ...), statusy; akceptacja lub odrzucenie z powodów przez biuro
Historia wersji	brak informacji	brak informacji	nie	tak, z historią zmian
Przechowywanie	chmura dostawcy	chmura dostawcy	chmura Adobe	własny serwer, szyfrowanie AES-256-GCM
Koszt	cennik niepubliczny	abonament firmy	bezpłatny, funkcje premium płatne	bez opłat (własne wdrożenie)

Źródło: opracowanie własne na podstawie [4, 5, 6, 7]

4 Opis architektury

System składa się z aplikacji klienckiej działającej w przeglądarce (PWA), serwera REST API, relacyjnej bazy danych oraz magazynu obiektów na pliki dokumentów. Wszystkie komponenty działają w kontenerach Docker i są uruchamiane wspólnie przez Docker Compose.

Rysunek 1 przedstawia kontenery systemu w notacji modelu C4. Użytkownik łączy się wyłącznie z kontenerem frontend: serwer nginx udostępnia zbudowaną aplikację React, a żądania na ścieżki /api/* przekazuje do kontenera backend. Przeglądarka łączy się z jednym adresem (ta sama domena dla aplikacji i API) i nie trzeba konfigurować mechanizmu CORS. Backend zapisuje dane w bazie PostgreSQL i zaszyfrowane pliki w magazynie MinIO, zgodnym z interfejsem Amazon S3. Prometheus okresowo pobiera metryki backendu, a Grafana prezentuje je na dashboardach. Administrator nadzoruje system.



Rysunek 1: Diagram kontenerów systemu (model C4)

Źródło: opracowanie własne

Kontenery są rozdzielone na trzy sieci Dockera według zasady najmniejszych uprawnień: app (nginx i backend), data (backend, PostgreSQL, MinIO) oraz monitoring (backend, Pro-

metheus, Grafana). Frontend nie ma bezpośredniego dostępu do bazy danych ani do magazynu plików, a jedynym kontenerem obecnym we wszystkich sieciach jest backend. Szczegóły konfiguracji opisano w podrozdziale 5.4.

4.1 Wybór technologii

W całym projekcie użyto języka TypeScript. Frontend i backend korzystają dzięki temu z tych samych typów i schematów walidacji, umieszczonych we wspólnym pakiecie. Zestawienie technologii zawiera tabela 2.

Tabela 2: Technologie użyte w projekcie

Obszar	Technologia	Uzasadnienie
Backend	Node.js 22, Express	Wskazany w wymaganiach, prosty serwer REST
Baza danych	PostgreSQL 16	Wskazana w wymaganiach
ORM i migracje	Prisma	Schemat bazy w jednym pliku, migracje wersjonowane w repozytorium
Pliki	MinIO (S3)	Magazyn obiektów w osobnym konterze
Walidacja	Zod	Te same reguły na froncie i w API
Frontend	React 19, Vite	Szybkie środowisko deweloperskie, łatwa konfiguracja PWA
Uwierzytelnianie	JWT, argon2	Tokeny z wymagań; zalecana funkcja skrótu haseł
Szyfrowanie	AES-256-GCM (node:crypto)	Szyfrowanie kopertowe plików przed zapisem
Monitoring	Prometheus, Grafana	Metryki, stan zdrowia, alerty, dashboardy
Konteneryzacja	Docker, Docker Compose	Uruchomienie całego systemu jednym poleceniem
Testy	Vitest, Supertest, Playwright	Testy funkcjonalne API i interfejsu, testy jednostkowe

Źródło: opracowanie własne

4.2 Struktura repozytorium

Kod jest zorganizowany jako monorepo oparte na mechanizmie *workspaces* menedżera npm. Jedno polecenie `npm install` instaluje zależności wszystkich pakietów, a wspólne skrypty (`build`, `lint`, `test`) uruchamiane są z katalogu głównego. Podział katalogów przedstawia tabela 3.

Pakiet `shared` jest kompilowany jako pierwszy, ponieważ importują go backend i frontend. Jakość kodu pilnują ESLint (reguły dla TypeScriptu i Reacta) oraz Prettier (jednolite formatowanie); oba narzędzia uruchamia polecenie `npm run lint`. Wspólne ustawienia kompilatora

Tabela 3: Katalogi repozytorium

Katalog	Zawartość
backend/	Serwer REST API (Express)
frontend/	Aplikacja kliencka React (PWA)
shared/	Schematy walidacji, typy i wyliczenia wspólne dla frontu i backendu
e2e/	Funkcjonalne testy interfejsu (Playwright)
monitoring/	Konfiguracja Prometheusa, Grafany i alertów
docs/	Niniejsze sprawozdanie ($\text{\LaTeX}$) i diagramy
scripts/	Skrypty pomocnicze, m.in. budowanie sprawozdania w Dockerze

Źródło: opracowanie własne

TypeScript (tryb `strict`) znajdują się w pliku `tsconfig.base.json`, rozszerzanym przez każdy pakiet.

5 Opis implementacji

Kolejne podrozdziały opisują frontend, backend, bazę danych, konteneryzację, testy i monitoring.

5.1 Frontend

Frontend to aplikacja jednostronicowa (SPA) napisana w React 19 i TypeScriptie, budowana narzędziem Vite. W Dockerze gotowe pliki serwuje nginx (podrozdział 5.4). Aplikacja komunikuje się z API pod tym samym adresem (/api) i nie wymaga konfiguracji CORS.

5.1.1 Struktura aplikacji

- `api/`: klient HTTP (dodaje token, zamienia odpowiedzi błędów na wyjątek `ApiError` z kodem i błędami pól) oraz funkcje wywołujące poszczególne endpointy;
- `auth/`: stan zalogowania (kontekst Reacta), zapis sesji i strażnicy tras;
- `pages/`: ekrany aplikacji; `layout/`: układ stron; `components/`: wspólne elementy (pole formularza, komunikat);
- `i18n/texts.ts`: wszystkie teksty interfejsu po polsku w jednym module; komponenty nie zawierają napisów na stałe.

Nawigacją zarządza React Router, a adresy są po polsku (/logowanie, /rejestracja, /profil). Dane z serwera pobiera i przechowuje w pamięci podręcznej biblioteka TanStack Query. Błędy klienta (4xx) nie są ponawiane, bo ponowienie dałoby tę samą odpowiedź, a błędy sieci i serwera są ponawiane jeden raz.

5.1.2 Logowanie i sesja

Po zalogowaniu token JWT, dane użytkownika i czas wygaśnięcia tokenu są zapisywane w `localStorage`, dzięki czemu sesja przetrwa odświeżenie strony i ponowne uruchomienie aplikacji zainstalowanej jako PWA. Przy starcie zapisana sesja jest sprawdzana zapytaniem `GET /api/auth/me`. Każda odpowiedź 401 na żądanie z tokenem (token wygasł albo konto usunięto) wylogowuje użytkownika z komunikatem „Sesja wygasła. Zaloguj się ponownie.”. Aplikacja wylogowuje też sama w chwili wygaśnięcia tokenu. Przechowywanie tokenu w `localStorage` oznacza, że skrypt wstrzyknięty na stronę (atak XSS) mógłby go odczytać. Ryzyko ogranicza React, który domyślnie zabezpiecza wyświetlane dane, brak zewnętrznych skryptów i krótki czas życia tokenu (1 godzina). Alternatywą byłoby ciasteczko `HttpOnly`, które wymaga jednak dodatkowej ochrony przed atakami CSRF.

Strażnik `RequireAuth` przekierowuje niezalogowanego użytkownika do ekranu logowania i zapamiętuje adres, pod który chciał wejść, a po zalogowaniu kieruje go z powrotem pod ten adres. Strażnik `GuestOnly` nie pokazuje ekranów logowania i rejestracji osobie już zalogowanej.

5.1.3 Formularze

Formularze obsługuje biblioteka `react-hook-form` ze schematami `Zod` z pakietu `shared`. Te same reguły i te same polskie komunikaty obowiązują w przeglądarce i w API. Niepoprawny formularz nie jest nawet wysyłany, a błędy pojawiają się przy polach. Błędy, które zna tylko serwer (np. zajęty login, odpowiedź 409), są przypisywane do właściwych pól na podstawie `details` z odpowiedzi API. Po rejestracji aplikacja automatycznie loguje użytkownika. Pola formularza są opisane etykietami, a pole z błędem ma atrybut `aria-invalid` i powiązany komunikat (`aria-describedby`), dzięki czemu błąd odczytują też czytniki ekranu.

5.1.4 Układ dla urządzeń mobilnych

Style są pisane w podejściu *mobile first*: podstawowe reguły dotyczą telefonów, a szersze ekrany dostają poprawki w `media query`. Na telefonie nawigacja główna jest paskiem przyklejonym do dolnej krawędzi ekranu, w zasięgu kciuka. Wszystkie elementy dotykowe (przyciski, pola, odnośniki nawigacji) mają co najmniej 48 pikseli wysokości, więcej niż zalecane minimum 44 pikseli. Układ uwzględnia wycięcia ekranu (`env(safe-area-inset-*)`). Na ekranie o szerokości od 768 pikseli nawigacja przechodzi pod górny pasek.

5.1.5 Dokumenty i profil

Lista dokumentów. Ekran startowy pokazuje dokumenty użytkownika jako karty z tytułem, kolorowym znacznikiem statusu (kolor i napis, żeby informacja nie zależała tylko od koloru), typem, numerem, numerem wersji i datą ostatniej zmiany. Nad listą znajdują się filtry typu i statusu oraz pole wyszukiwania w tytule i numerze. Wartości filtrów są zapisane w adresie strony (np. `/?type=WZ&status=SUBMITTED`). Odświeżenie strony, przycisk „wstecz” i udostępniony odnośnik zachowują dzięki temu widok. Zapytanie wyszukiwania jest wysyłane 300 ms po ostatnim naciśnięciu klawisza, nie po każdym znaku. Przy przełączaniu stron listy poprzednie wyniki pozostają widoczne do czasu wczytania nowych i ekran nie miga.

Dodawanie dokumentu. Formularz zawiera typ, tytuł, opcjonalny numer i uwagi oraz plik. Przeglądarka od razu sprawdza rozmiar i zadeklarowany typ pliku, żeby użytkownik nie czekał na wysłanie 10 MB, które i tak zostałyby odrzucone. Ostatecznie o formacie decyduje jednak serwer, sprawdzając zawartość: plik wykonywalny przemianowany na `.pdf` przejdzie kontrolę w przeglądarce, ale komunikat serwera pojawi się przy polu pliku.

Szczegóły dokumentu. Ekran szczegółów pokazuje podgląd bieżącej wersji, przyciski zmiany statusu, formularz edycji tytułu i numeru, listę wersji z przyciskiem pobierania, formularz nowej wersji, historię zmian i przycisk usunięcia. Plik wymaga nagłówka `Authorization`, dlatego nie można go podać bezpośrednio w `<img src>`. Aplikacja pobiera go jako obiekt `Blob` i wyświetla przez tymczasowy adres `blob:`, który zwalnia, gdy przestaje być potrzebny. Zdjęcia są pokazywane bezpośrednio. Dla plików PDF wyświetlana jest karta z przyciskiem „Otwórz PDF”, bo większość przeglądarek mobilnych nie wyświetla PDF osadzonego w stronie. Przyciski zmiany statusu powstają z tych samych reguł przejść, które sprawdza API (pakiet `shared`). Interfejs pokazuje przez to tylko dozwolone zmiany. Zmiana statusu i usunięcie wymagają potwierdzenia. Dokument zarchiwizowany jest wyświetlany tylko do odczytu. Po każdej zmianie dane dokumentu i listy są odświeżane automatycznie (unieważnienie pamięci podręcznej `TanStack Query`).

Profil kierowcy. Formularz profilu korzysta ze wspólnego schematu, a po zapisaniu pokazuje wartości znormalizowane przez serwer, np. numer telefonu bez spacji. Pola mają atrybuty `autocomplete` i `inputmode` (np. klawiatura numeryczna dla telefonu), co ułatwia wypełnianie na telefonie.

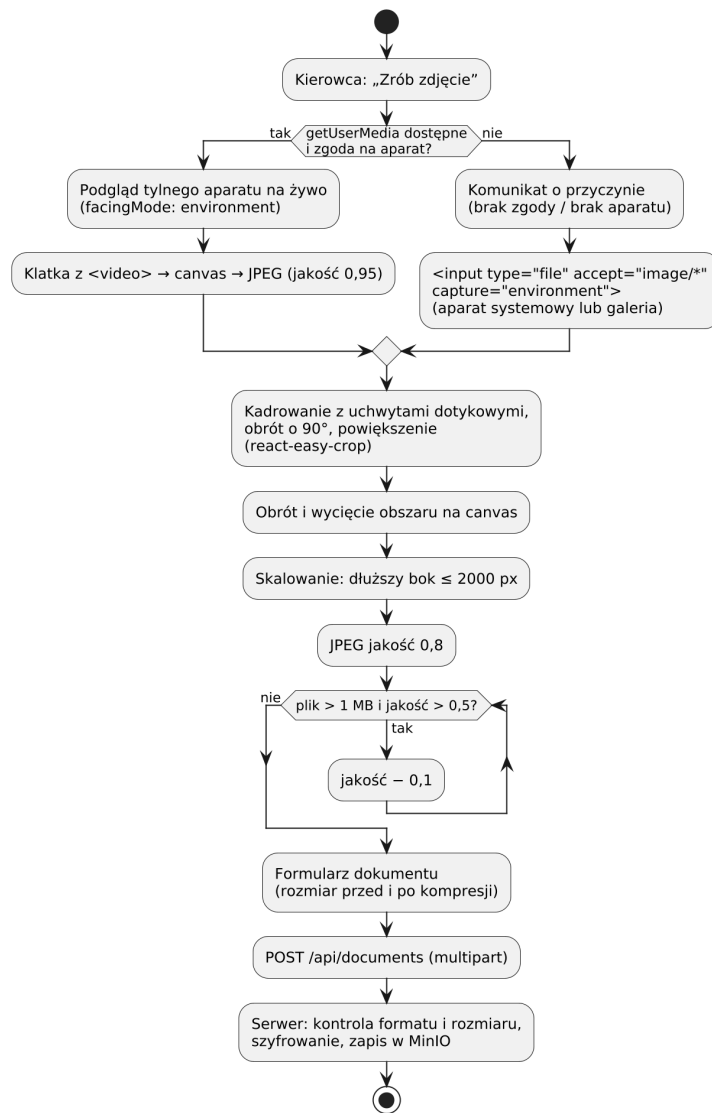
5.1.6 Zdjęcie dokumentu aparatem

Przebieg od zdjęcia do zapisu dokumentu przedstawia rysunek 2. Pole pliku w formularzu dokumentu (i w formularzu nowej wersji) ma przycisk „Zrób zdjęcie”, który otwiera pełnoekranowy widok aparatu.

Aparat. Hook `useCamera` uruchamia strumień wideo funkcją `navigator.mediaDevices.getUserMedia` [8] z ograniczeniem `facingMode: environment` (tylny aparat telefonu) i preferowaną rozdzielczością Full HD, a następnie podłącza go do elementu `<video>`. Duży okrągły przycisk migawki kopiuje bieżącą klatkę na `canvas` i zapisuje ją jako JPEG w pełnej rozdzielczości aparatu; kompresja następuje dopiero po kadrowaniu. Przy wyjściu z widoku aparatu wszystkie ścieżki strumienia są zatrzymywane i aparat (razem z jego lampką) wyłącza się od razu. `getUserMedia` działa tylko w bezpiecznym kontekście (HTTPS albo `localhost`). Gdy funkcja jest niedostępna, użytkownik odmówił dostępu albo urządzenie nie ma aparatu, aplikacja wyjaśnia przyczynę i oferuje zapasowe pole wyboru pliku, które na telefonie otwiera systemową aplikację aparatu lub galerię:

```
<input type="file" accept="image/*" capture="environment">
```

Kadrowanie. Zdjęcie trafia do edytora opartego na bibliotece `react-easy-crop`: obszar kadru przesuwany palcem, powiększa gestem „szczypania” albo suwakiem, a przycisk „Obróć o 90°” obraca zdjęcie. Biblioteka podaje obszar kadru we współrzędnych obróconego obrazu, dlatego



Rysunek 2: Przepływ zdjęcia: aparat, kadrowanie, kompresja, wysłanie

Źródło: opracowanie własne

obraz jest najpierw rysowany obrócony na płótnie o rozmiarze obróconego prostokąta, a dopiero potem wycinany jest wybrany fragment.

Kompresja. Wykadrowane zdjęcie jest skalowane tak, by dłuższy bok miał najwyżej 2000 pikseli (z zachowaniem proporcji, bez powiększania mniejszych zdjęć), i kodowane jako JPEG o jakości 0,8. Jeśli plik przekracza 1 MB, jakość jest obniżana o 0,1, ale nie poniżej 0,5. Taką samą kompresję przechodzą duże zdjęcia JPG i PNG wybrane z galerii telefonu. Limit 10 MB jest sprawdzany dopiero po kompresji, bo zdjęcie z nowoczesnego telefonu bywa większe, a po kompresji mieści się w nim z zapasem. Pliki PDF są wysyłane bez zmian. Formularz pokazuje rozmiar przed kompresją i po niej. Wyniki pomiaru dla typowych zdjęć 12 Mpx zawiera tabela 4.

Tabela 4: Kompresja zdjęć przed wysłaniem (pomiar w przeglądarce Chromium)

Zdjęcie wejściowe	Rozmiar	Po kompresji	Rozmiar
4000 × 3000 px (poziome)	3,7 MB	2000 × 1500 px	778 kB
3024 × 4032 px (pionowe)	3,7 MB	1500 × 2000 px	708 kB

Źródło: opracowanie własne

Obliczenia niezależne od przeglądarki (rozmiar po skalowaniu, rozmiar po obrocie, kolejne poziomy jakości, decyzja, czy kompresować) są wydzielone do modułu `image-math.ts` i pokryte testami jednostkowymi. Operacje na płótnie są sprawdzane testami Playwright w prawdziwej przeglądarce.

5.1.7 Aplikacja instalowalna (PWA)

Aplikacja jest progresywną aplikacją webową (PWA). Wtyczka `vite-plugin-pwa` generuje podczas budowania manifest i service worker (biblioteka Workbox).

Manifest. Plik `manifest.webmanifest` opisuje aplikację do instalacji: nazwę („Aplikacja kierowcy – dokumenty przewozowe”, skrót „Kierowca”), język polski, adres startowy, tryb `standalone` (aplikacja uruchamia się bez paska adresu, jak aplikacja natywna), orientację pionową, kolory motywu oraz ikony 192 i 512 pikseli. Osobna ikona *maskable* ma rysunek w bezpiecznym obszarze 80 %, dzięki czemu Android może ją przyciąć do kształtu ikon systemu. Ikony są generowane z jednego pliku SVG skryptem, który renderuje je w przeglądarce Chromium.

Service worker. Service worker przechowuje w pamięci podręcznej wyłącznie pliki aplikacji (HTML, JavaScript, CSS, ikony). Wystarczają one, żeby aplikacja uruchomiła się także bez połączenia z siecią. Adresy aplikacji otwierane bez sieci są obsługiwane przez zapisany `index.html`, a routowaniem zajmuje się React Router. Żądania do API (`/api/*`), w tym pliki dokumentów, mają strategię *NetworkOnly*: nie są nigdy zapisywane na urządzeniu, bo dokumenty są poufne,

a zapisane kopie przetrwałyby wylogowanie. Po wdrożeniu nowej wersji service worker aktualizuje się automatycznie. Serwer nginx wysyła `sw.js` i manifest z nagłówkiem wymuszającym sprawdzenie aktualności, żeby zainstalowane aplikacje nie utknęły na starej wersji.

Brak połączenia. Gdy przeglądarka zgłasza brak sieci, u góry ekranu pojawia się baner „Brak połączenia z internetem. Możesz przeglądać aplikację, ale dokumenty wymagają połączenia.”, a po odzyskaniu połączenia znika. Błędy żądań wykonanych bez sieci są pokazywane jako komunikat o braku połączenia.

Weryfikacja. Plan zakładał sprawdzenie aplikacji narzędziem Lighthouse. Kategoria „PWA” została jednak usunięta z Lighthouse w wersji 12, dlatego wymagania instalowalności sprawdzają testy Playwright (podrozdział 5.5). Instalację sprawdzono też ręcznie na prawdziwym telefonie z Androidem (rysunek 10).

5.1.8 Panel biura

Panel biura jest częścią tej samej aplikacji, dostępną pod adresem `/biuro` dla kont z rolą biura (podrozdział 5.2.11). Jest przeznaczony głównie do pracy w przeglądarce na komputerze: szerszy układ, tabela dokumentów i dwie kolumny na ekranie dokumentu. Na telefonie tabela zamienia się w karty, a nawigacja w dolny pasek, tak jak w części kierowcy.

Lista. Widok „Do sprawdzenia” pokazuje dokumenty przesłane przez wszystkich kierowców, a widok „Wszystkie dokumenty” dodatkowo filtr statusu. Kolumny to kierowca (imię i nazwisko z profilu, login i firma), tytuł, typ, numer, status, numer wersji i data zmiany. Filtry kierowcy, typu i tekstu są zapisane w adresie strony, tak jak na liście kierowcy.

Dokument i decyzja. Ekran dokumentu pokazuje duży podgląd bieżącej wersji, a obok dane kontaktowe kierowcy i formularz decyzji: pole „Komentarz dla kierowcy” oraz przyciski „Akceptuj” i „Odrzuć”. Formularz sprawdza wspólnym schematem, czy odrzucenie ma powód, zanim wyśle żądanie. Poniżej znajdują się wersje pliku do pobrania i historia z komentarzami biura. Biuro nie ma przycisków edycji ani usuwania.

Strona kierowcy. Kierowca nie ma już przycisku akceptacji. Po przesłaniu dokumentu widzi komunikat, że dokument czeka na decyzję biura. Odrzucony dokument ma czerwoną ramkę z powodem podanym przez biuro, a historia pokazuje, kto z biura podjął decyzję. Kierowca może dodać nową wersję (np. nowe zdjęcie) i przesłać dokument ponownie.

Rozdzielenie ról. Strażnik tras `RequireRole` kieruje każdą rolą do jej części aplikacji. Biuro po zalogowaniu trafia na `/biuro`, a kierowca próbujący otworzyć `/biuro` wraca na swoją listę.

Zapisana sesja sprzed wprowadzenia ról (bez poprawnej roli) jest odrzucana. Użytkownik loguje się wtedy ponownie i dostaje aktualne dane konta.

Aktualność danych. Biblioteka TanStack Query domyślnie uznawała pobrane dane za aktualne przez 30 sekund. Dopóki dokument zmieniał tylko jego właściciel, to wystarczało, bo jego własne zmiany odświeżały pamięć podręczną. Teraz zmieniają go dwie strony, dlatego listy i szczegóły dokumentów są pobierane na nowo przy każdym wejściu na ekran. Błąd, który wynikał z poprzedniego ustawienia, wykrył test E2E (podrozdział 5.5).

Rysunek 3 pokazuje panel biura w przeglądarce na komputerze (okno 1440×900 punktów, zrzuty wykonane skryptem `scripts/report-screenshots.mjs`).

5.1.9 Wygląd aplikacji

Rysunki 4 i 5 pokazują ekrany części kierowcy na prawdziwym telefonie (Samsung Galaxy A26, Chrome). Zrzuty wykonano poleceniem `adb exec-out screencap` na jednorazowej kopii stosu Docker z danymi demonstracyjnymi (konto `jan_kowalski`) i nie zawierają prawdziwych danych. Fotografowany dokument to fikcyjny list przewozowy CMR wyświetlony na monitorze komputera.

5.2 Backend

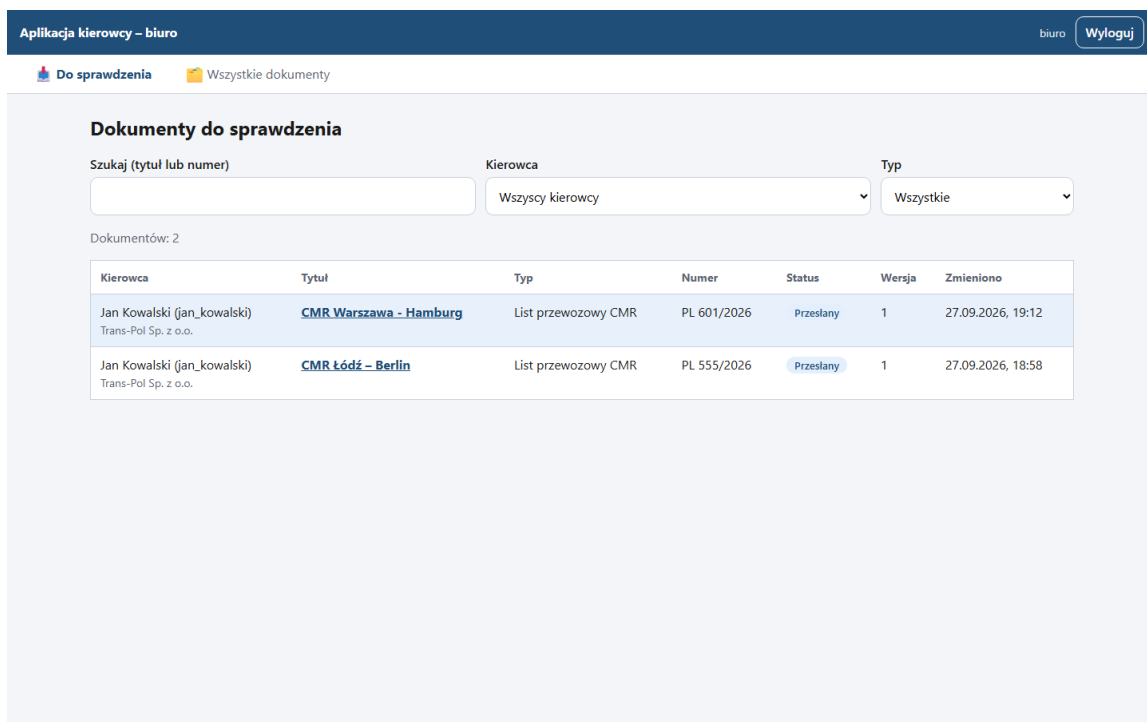
Backend to serwer REST API napisany w TypeScriptie na platformie Node.js z frameworkiem Express 5. Kod jest kompilowany do modułów ECMAScript i uruchamiany w kontenerze opisanym w podrozdziale 5.4.

5.2.1 Warstwy aplikacji

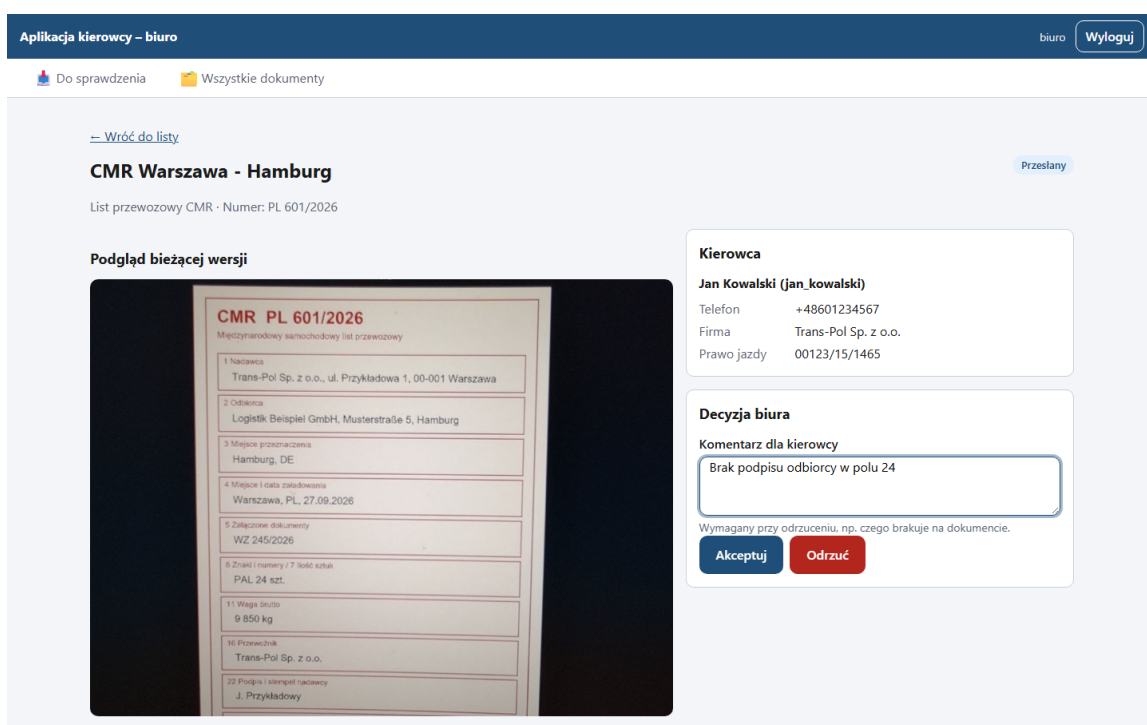
Kod podzielono na warstwy, z których każda korzysta tylko z warstwy leżącej bezpośrednio pod nią:

- **trasy** (`routes/`) przypisują metody i ścieżki HTTP do funkcji kontrolera;
- **kontrolery** (`controllers/`) odczytują dane z żądania, wywołują serwis i budują odpowiedź HTTP (kod statusu, treść JSON);
- **serwisy** (`services/`) zawierają logikę biznesową i nie znają protokołu HTTP;
- **repozytoria** (`repositories/`) jako jedyne komunikują się z bazą danych (Prisma) i magazynem plików (API S3).

Aplikację buduje funkcja `createApp(deps)`, która otrzymuje gotowe zależności: konfigurację, logger, klienta bazy danych i klienta S3. Tworzy je dopiero plik `server.ts`, który



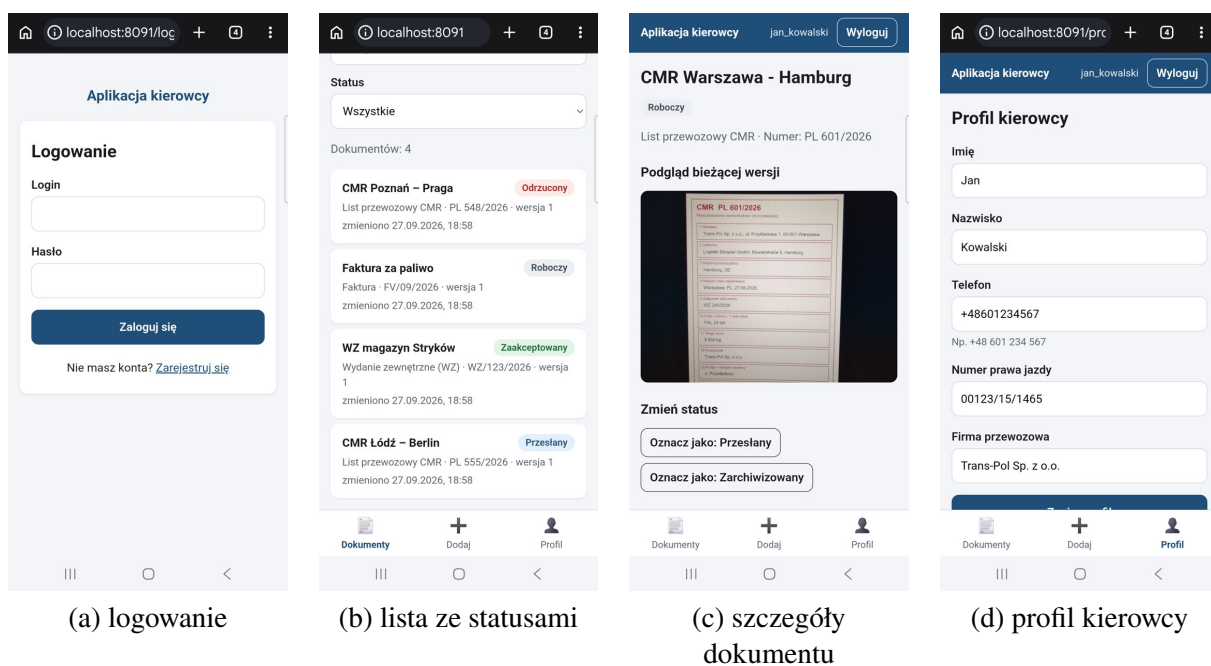
(a) lista dokumentów do sprawdzenia



(b) dokument z danymi kierowcy i formularzem decyzji

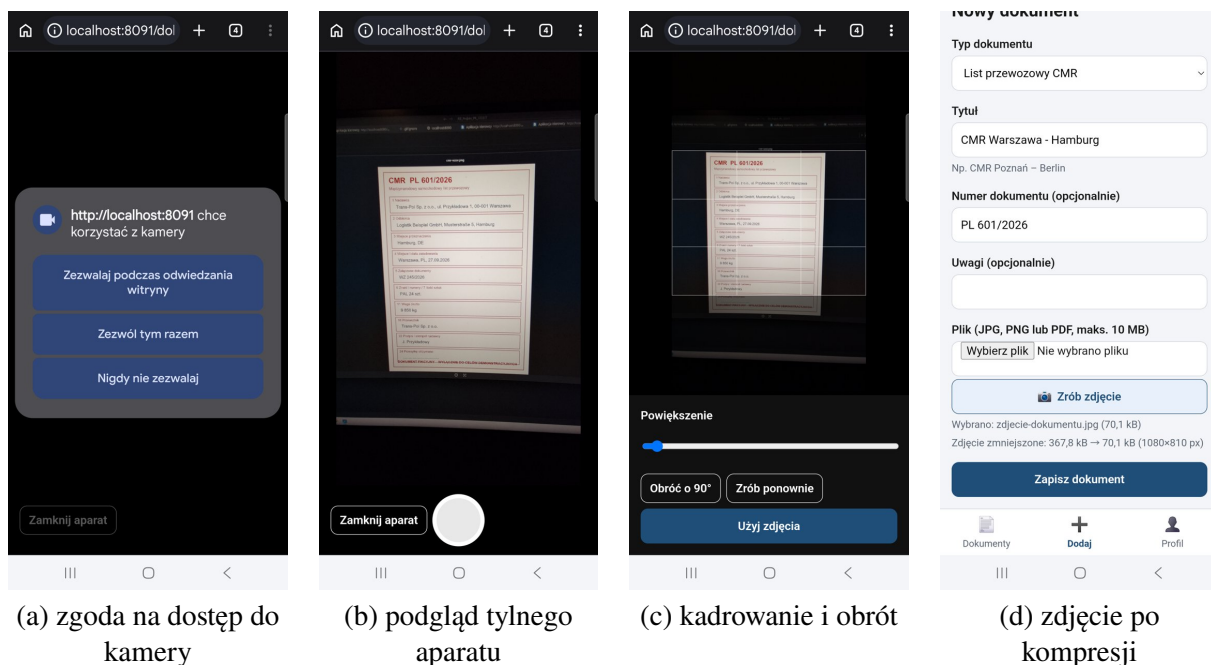
Rysunek 3: Panel biura w przeglądarce na komputerze

Źródło: opracowanie własne (zrzuty ekranu aplikacji)



Rysunek 4: Ekrany kierowcy na telefonie: logowanie, lista, szczegóły dokumentu i profil

Źródło: opracowanie własne (zrzuty ekranu telefonu)



Rysunek 5: Dodawanie dokumentu zdjęciem na telefonie

Źródło: opracowanie własne (zrzuty ekranu telefonu)

następnie uruchamia serwer HTTP. Dzięki temu testy mogą zbudować aplikację z innymi zależnościami, np. z adresem nieistniejącej bazy danych, i sprawdzić zachowanie systemu przy awarii (podrozdział 5.5).

5.2.2 Walidacja konfiguracji

Przy starcie zmienne środowiskowe są sprawdzane schematem biblioteki Zod. Sprawdzane są m.in.: format adresu bazy danych, poprawność adresu URL magazynu MinIO, minimalna długość sekretu JWT (32 znaki) oraz to, czy klucz główny szyfrowania jest zapisem base64 dokładnie 32 bajtów. Puste wartości są traktowane jak brakujące. Przy błędnej konfiguracji proces wypisuje listę *wszystkich* nieprawidłowych zmiennych (bez ich wartości, żeby nie ujawnić sekretów w logach) i kończy się kodem 1. Kontener z błędną konfiguracją zatrzymuje się od razu z opisem przyczyny i nie czeka z błędem do pierwszego żądania.

5.2.3 Obsługa błędów

Wszystkie błędy trafiają do jednego middleware'u obsługi błędów, który zwraca je w jednolitym formacie:

```
{ "error": { "code": "VALIDATION_ERROR",
            "message": "Request validation failed",
            "details": [ { "path": "email", "message": "..." } ] } }
```

Pole code jest stałym identyfikatorem, na podstawie którego frontend wyświetla komunikat po polsku. Pole message jest opisem dla programisty, a details zawiera np. błędy poszczególnych pól formularza. Błędy przewidziane przez aplikację są klasami pochodnymi `AppError` (tabela 5). Każdy inny wyjątek zamieniany jest na odpowiedź 500 `INTERNAL_ERROR` z ogólnym komunikatem, a szczegóły (np. treść błędu bazy danych) trafiają wyłącznie do logu. Express 5 przekazuje do tego middleware'u również wyjątki z funkcji asynchronicznych, dlatego kontrolery nie potrzebują bloków `try/catch`.

5.2.4 Logowanie zdarzeń

Logi są zapisywane biblioteką `pino` jako jeden obiekt JSON na linię. Taki format łatwo przeszukiwać i zbierać centralnie (podrozdział 5.6). Każde żądanie dostaje identyfikator (`requestId`): jeśli klient przesłał poprawny nagłówek `X-Request-Id`, jest on używany ponownie, w przeciwnym razie generowany jest nowy UUID. Identyfikator trafia do nagłówka odpowiedzi i do każdego wpisu w logu powstałego podczas obsługi żądania. Po zakończeniu żądania zapisywany jest jeden wpis z metodą, adresem, kodem odpowiedzi i czasem obsługi, na poziomie `warn` dla błędów klienta i `error` dla błędów serwera. Nagłówki `Authorization` i `Cookie` są w logach zamaskowane. Zapytania kontrolne Dockera do `/health`, wykonywane

Tabela 5: Kody błędów API

Kod	HTTP	Znaczenie
VALIDATION_ERROR	400	Niepoprawne dane wejściowe (<code>ValidationError</code>)
INVALID_JSON	400	Treść żądania nie jest poprawnym JSON-em
UNAUTHORIZED	401	Brak lub niepoprawny token dostępu
TOKEN_EXPIRED	401	Token dostępu wygasł
INVALID_CREDENTIALS	401	Zły login lub hasło
FORBIDDEN	403	Brak uprawnień do operacji (<code>ForbiddenError</code>)
NOT_FOUND	404	Nieznana ścieżka lub zasób (<code>NotFoundError</code>)
CONFLICT	409	Dane kolidują z istniejącymi, np. zajęty login (<code>ConflictError</code>)
PAYLOAD_TOO_LARGE	413	Treść JSON większa niż 1 MB
TOO_MANY_REQUESTS	429	Przekroczony limit nieudanych logowań
INVALID_STATUS_TRANSITION	409	Niedozwolona zmiana statusu dokumentu
DOCUMENT_ARCHIVED	409	Próba zmiany zarchiwizowanego dokumentu
FILE_INTEGRITY_ERROR	500	Plik w magazynie nie przeszedł kontroli integralności
INTERNAL_ERROR	500	Nieoczekiwany błąd serwera

Źródło: opracowanie własne

co 10 sekund, nie są logowane. W trybie deweloperskim logi są formatowane czytelnie przez `pino-pretty`.

5.2.5 Punkty kontroli stanu

Backend udostępnia dwa punkty kontroli stanu (tabela 6), dostępne pod ścieżką `/health` (dla Dockera) oraz `/api/health` (przez serwer `nginx`).

- **Liveness** (`GET /health`) odpowiada kodem 200, dopóki proces obsługuje żądania HTTP. Celowo nie sprawdza bazy danych: awaria bazy nie powinna powodować restartu działającego backendu.
- **Readiness** (`GET /health/ready`) równolegle sprawdza PostgreSQL (zapytanie `SELECT 1`) i MinIO (`HeadBucket` na buckecie dokumentów), każde z limitem 3 sekund. Zwraca 200 i `{"status": "ready"}` albo 503 i `{"status": "not_ready"}`, a w polu `checks` status każdej zależności (`up` lub `down`). Przyczyny awarii są zapisywane w logu, ale nie są zwracane klientowi, żeby nie ujawniać wewnętrznych adresów.

5.2.6 Rejestracja i logowanie

Reguły formularza rejestracji są zapisane raz, jako schemat `Zod` we wspólnym pakiecie `shared`. Ten sam schemat sprawdza dane w API i w formularzu w przeglądarce, dzięki czemu reguły po obu stronach są zawsze takie same. Wymagania to: login 3–30 znaków złożony z liter, cyfr i podkreślenia; poprawny adres e-mail; hasło o długości 8–128 znaków z wielką literą,

Tabela 6: Endpointy API

Metoda	Ścieżka	Opis
GET	/health, /api/health	Liveness – proces działa
GET	/health/ready, /api/health/ready	Readiness – baza danych i MinIO dostępne
POST	/api/auth/register	Rejestracja konta
POST	/api/auth/login	Logowanie, zwraca token JWT
GET	/api/auth/me	Dane zalogowanego użytkownika (wymaga tokenu)
GET	/api/profile	Profil zalogowanego kierowcy (wymaga tokenu)
PUT	/api/profile	Zmiana profilu (wymaga tokenu)
GET	/api/documents	Lista dokumentów z filtrami i stronicowaniem
POST	/api/documents	Dodanie dokumentu z plikiem (multipart)
GET	/api/documents/:id	Szczegóły z wersjami i historią
PATCH	/api/documents/:id	Zmiana tytułu, numeru, statusu
DELETE	/api/documents/:id	Usunięcie miękkie
POST	/api/documents/:id/versions	Nowa wersja pliku
GET	/api/documents/:id/versions/:no/file	Pobranie odszyfrowanego pliku
GET	/api/office/documents	Biuro: dokumenty wszystkich kierowców z filtrami
GET	/api/office/documents/:id	Biuro: szczegóły dokumentu z danymi kierowcy
GET	/api/office/documents/:id/versions/:no/file	Biuro: pobranie odszyfrowanego pliku
POST	/api/office/documents/:id/review	Biuro: akceptacja albo odrzucenie z powodem
GET	/api/office/drivers	Biuro: lista kierowców (filtr)
GET	/api/docs	Dokumentacja API (Swagger UI, OpenAPI 3.0)

Źródło: opracowanie własne

cyfrą i znakiem specjalnym oraz identyczne potwierdzenie hasła. Komunikaty są po polsku, bo wyświetla je interfejs. Login i adres e-mail są zamieniane na małe litery, dlatego Jan_K i jan_k to to samo konto. Górny limit długości hasła chroni serwer przed kosztownym haszowaniem bardzo długich danych.

Rejestracja (POST /api/auth/register) najpierw sprawdza, czy login lub e-mail nie są zajęte. Jeśli są, zwraca 409 CONFLICT z listą zajętych pól w details. Formularz pokazuje wtedy komunikat przy właściwym polu. Hasło jest zapisywane wyłącznie jako skrót algorytmem argon2id (64 MiB pamięci, 3 iteracje, losowa sól), rekomendowanym do przechowywania haseł [9]. Gdy dwie rejestracje tego samego loginu wystąpią jednocześnie, rozstrzyga ograniczenie unikalności w bazie danych, a przegrana rejestracja również dostaje 409.

Przebieg logowania przedstawia rysunek 6. Po poprawnym sprawdzeniu hasła serwer wystawia token JWT [10] podpisany algorytmem HS256 z kluczem JWT_SECRET. Token zawiera identyfikator użytkownika (sub), wystawcę (iss), odbiorcę (aud) i czas wygaśnięcia (exp, domyślnie 1 godzina). Zły login i złe hasło dają identyczną odpowiedź 401 INVALID_CREDENTIALS. Dla nieistniejącego loginu serwer i tak wykonuje weryfikację argon2 na fikcyjnym skrócie. Po czasie odpowiedzi nie da się zatem ustalić, które loginy istnieją.

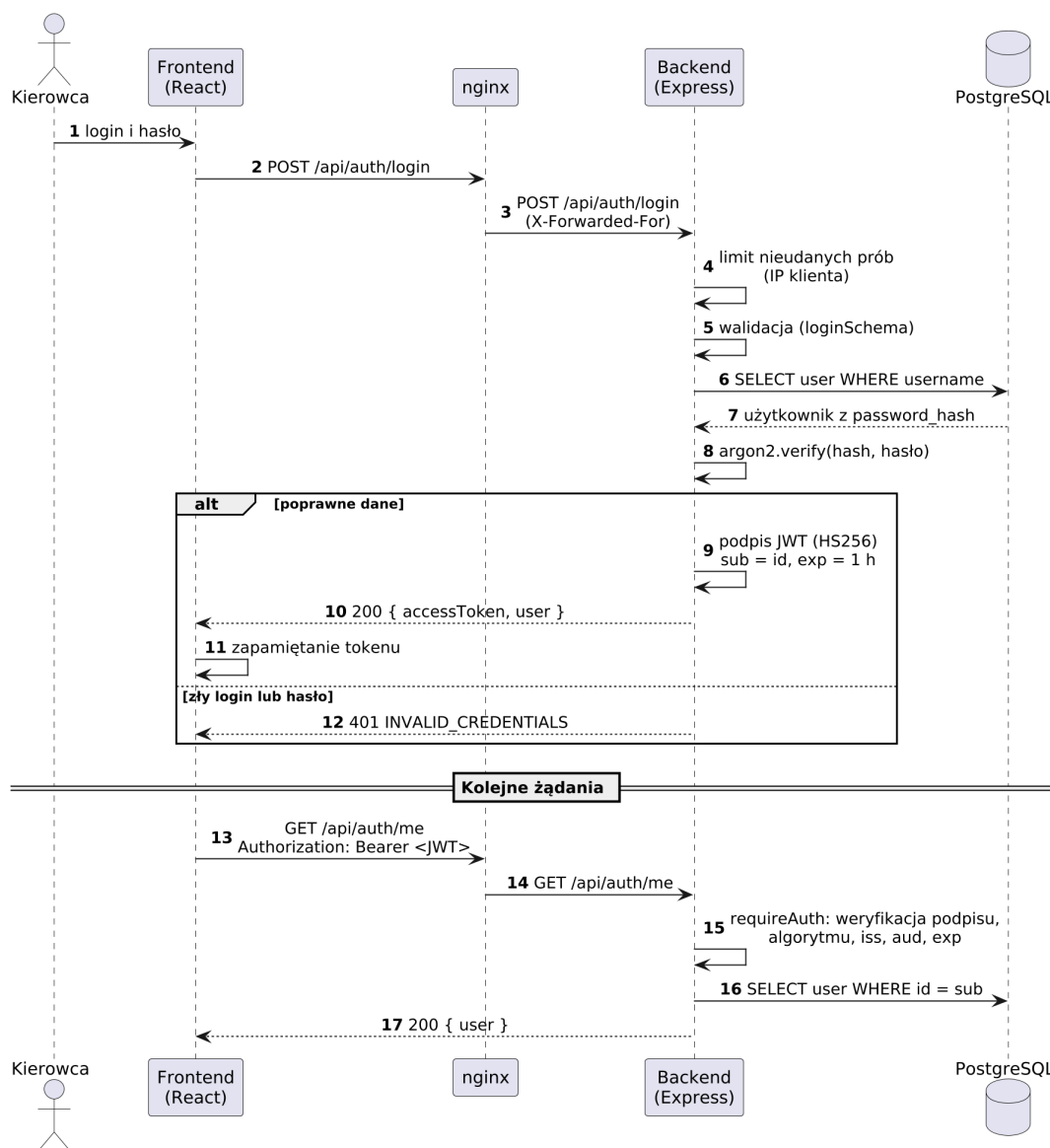
Chronione endpointy przechodzą przez middleware requireAuth. Oczekuje on nagłówek Authorization: Bearer <token> i weryfikuje podpis, dopuszczalny algorytm (wyłącznie HS256, co wyklucza m.in. niepodpisane tokeny alg: none), wystawcę, odbiorcę i czas ważności. Wygasły token daje kod TOKEN_EXPIRED, każdy inny problem kod UNAUTHORIZED. Endpoint /api/auth/me dodatkowo sprawdza, czy konto nadal istnieje.

5.2.7 Ochrona przed zgadywaniem haseł

Endpoint logowania ma limit nieudanych prób (biblioteka express-rate-limit): domyślnie 10 nieudanych prób z jednego adresu IP w ciągu 15 minut. Udań logowania nie są wliczane do limitu i nie przeszkadza on użytkownikowi, który podaje poprawne hasło. Po przekroczeniu limitu każda kolejna próba, nawet z poprawnym hasłem, dostaje 429 TOO_MANY_REQUESTS z nagłówkiem Retry-After, a zdarzenie jest zapisywane w logu. W Dockerze żądania przechodzą przez nginx, dlatego backend ufa jednemu serwerowi pośredniczącemu (TRUST_PROXY=1) i odczytuje adres klienta z nagłówka X-Forwarded-For. Bez tego wszyscy użytkownicy mieliby adres nginx i dzieliliby jeden limit. Liczniki są przechowywane w pamięci procesu, co wystarcza przy jednej instancji API.

5.2.8 Profil kierowcy

Każde konto ma dokładnie jeden profil kierowcy z imieniem, nazwiskiem, telefonem, numerem prawa jazdy i nazwą firmy przewozowej. Profil jest tworzony razem z kontem, w jednej operacji zapisu do bazy danych, dzięki czemu nie może powstać konto bez profilu. Endpoint /api/profile zawsze dotyczy profilu zalogowanego użytkownika, wskazanego przez token.



Rysunek 6: Sekwencja logowania i użycia tokenu JWT

Źródło: opracowanie własne

W adresie nie ma identyfikatora i nie da się nawet zaadresować cudzego profilu, a pola takie jak `userId` w treści żądania są ignorowane.

GET zwraca profil, a PUT zastępuje go w całości: pole pominięte, puste lub `null` jest czyszczone. Formularz profilu wysyła wszystkie pola naraz, a przy takim formularzu ta zasada jest prosta i jednoznaczna. Reguły walidacji znajdują się we wspólnym schemacie Zod. Imię i nazwisko mogą zawierać litery (także polskie), spację, myślnik i apostrof. Telefon to 9–15 cyfr z opcjonalnym znakiem +; spacje, myślniki i nawiasy wpisane dla czytelności są usuwane, np. `+48 601-234-567` zostaje zapisany jako `+48601234567`. Numer prawa jazdy jest zamieniany na wielkie litery. Jeśli konto zostało usunięte, a token jest jeszcze ważny, API odpowiada 401, tak samo jak `/api/auth/me`.

5.2.9 Dokumentacja API

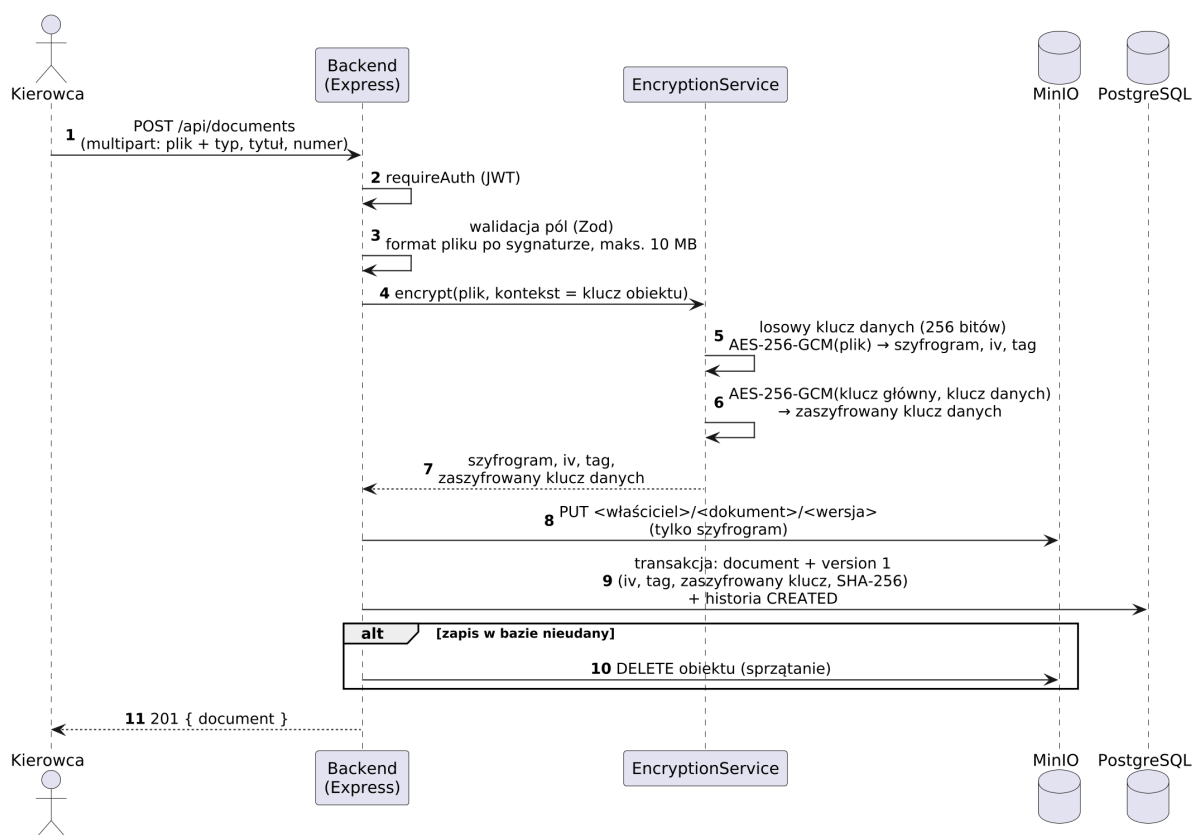
Specyfikacja API w formacie OpenAPI 3.0 jest dostępna pod adresem `/api/docs/openapi.json`, a interaktywna dokumentacja Swagger UI pod `/api/docs`. Można w niej wykonać żądania, także z tokenem JWT (przycisk *Authorize*). Schematy treści żądań rejestracji i logowania są generowane automatycznie z tych samych schematów Zod, które sprawdzają dane (z. `toJSONSchema`). Dokumentacja opisuje zatem dokładnie te reguły, które są stosowane. Schemat profilu jest opisany ręcznie, ponieważ jego walidacja najpierw normalizuje wartości (np. usuwa spacje z numeru telefonu), a taki schemat po przekształceniu traci w dokumentacji ograniczenia. Limity długości pochodzą jednak ze wspólnej stałej `PROFILE_LIMITS`. Test funkcjonalny sprawdza, czy specyfikacja zawiera oczekiwaną listę endpointów. Nowy endpoint trzeba dopisać do tej listy, co przypomina o uzupełnieniu dokumentacji.

5.2.10 Dokumenty

Dokument ma typ (CMR, WZ, faktura lub inny), status, tytuł, opcjonalny numer (np. numer listu CMR) i co najmniej jedną wersję pliku. Plik jest przesyłany jako formularz `multipart/form-data`, a jego treść przechowuje MinIO. Baza danych zawiera metadane, parametry szyfrowania i historię zmian.

Kontrola przesyłanych plików. Akceptowane są wyłącznie zdjęcia JPG i PNG oraz pliki PDF o rozmiarze do 10 MB. Format jest rozpoznawany po pierwszych bajtach pliku (sygnaturze formatu), a nie po rozszerzeniu nazwy ani po nagłówku `Content-Type`, które ustala klient i które łatwo sfałszować. Plik wykonywalny przemianowany na `.pdf` zostaje odrzucony. Błędy pól formularza i pliku są zgłaszane razem w jednej odpowiedzi 400, każdy przy swoim polu. Token jest sprawdzany przed odczytem pliku, dlatego anonimowe żądanie zostaje odrzucone, zanim serwer przyjmie 10 MB danych.

Szyfrowanie kopertowe. Każdy plik jest szyfrowany przed zapisem w MinIO (rysunek 7). Dla każdej wersji serwer losuje nowy 256-bitowy klucz danych i szyfruje nim plik algorytmem AES-256-GCM, który zapewnia jednocześnie poufność i integralność [11]. Klucz danych jest następnie szyfrowany („opakowywany”) kluczem głównym z konfiguracji (MASTER_ENCRYPTION_KEY) i zapisywany w bazie razem z wektorem inicjującym i znacznikiem uwierzytelniającym. Do MinIO trafia wyłącznie szyfrogram. Takie rozwiązanie ma trzy zalety. Kluczem głównym nie szyfruje się dużych ilości danych. Wyciek pojedynczego klucza danych ujawnia tylko jeden plik. Wymiana klucza głównego wymaga ponownego zaszyfrowania jedynie 32-bajtowych kluczy danych, bez przepisywania samych plików. Oba szyfrowania mają jako dane uwierzytelniane (AAD) klucz obiektu w MinIO. Szyfrogramu ani klucza danych skopiowanego do innej wersji nie da się odszyfrować.



Rysunek 7: Sekwencja dodania dokumentu z szyfrowaniem kopertowym

Źródło: opracowanie własne

Zapis przebiega w kolejności: szyfrowanie, zapis obiektu w MinIO, transakcja w bazie danych (dokument, wersja i wpis historii). Jeśli transakcja się nie powiedzie, obiekt jest usuwany z MinIO, żeby nie zostawały pliki bez metadanych.

Pobieranie i kontrola integralności. Endpoint `/api/documents/:id/versions/:no/file` odczytuje szyfrogram, odszyfrowuje klucz danych, a potem plik, i porównuje skrót SHA-256 z zapisanym w bazie. Dopiero po pozytywnej weryfikacji znacznika GCM i skrótu

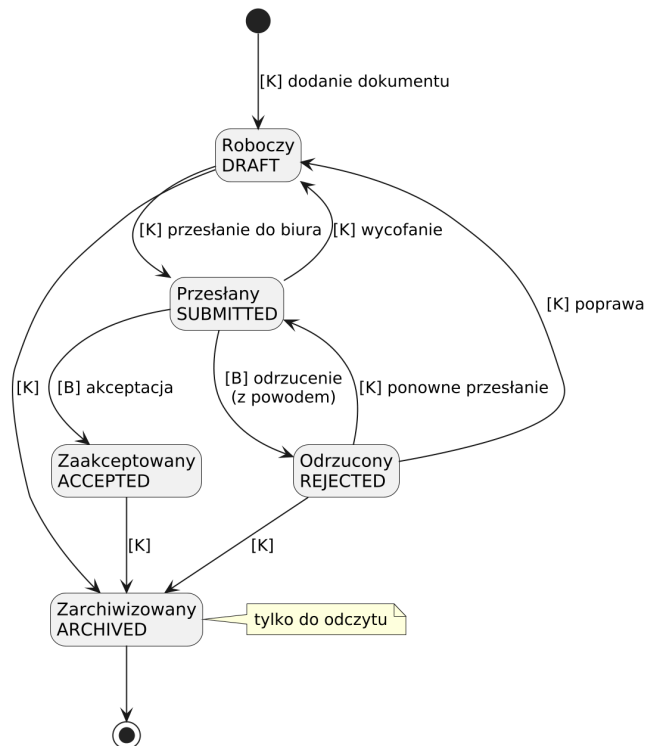
plik jest wysyłany do klienta. Plik zmieniony w magazynie choćby o jeden bajt powoduje odpowiedź 500 FILE_INTEGRITY_ERROR i wpis w logu, a klient nie otrzymuje żadnej części treści. Z tego powodu plik jest odszyfrowywany w całości w pamięci, a nie przesyłany strumieniowo: przy strumieniu część danych trafiłaby do klienta przed sprawdzeniem znacznika, który GCM weryfikuje na końcu. Przy limicie 10 MB takie podejście jest bezpieczne pamięciowo. Odpowiedź ma nagłówki Cache-Control: private, no-store (poufne dokumenty nie są zapisywane w pamięci podręcznej), X-Content-Type-Options: nosniff, ETag równy skrótowi SHA-256 oraz nazwę pliku w dwóch postaciach: bez polskich znaków (filename) i w UTF-8 (filename*).

Wersjonowanie. Endpoint POST /api/documents/:id/versions dodaje nową wersję pliku. Numer wersji jest zwiększany w bazie danych (`current_version = current_version + 1`) w tej samej transakcji, w której zapisywana jest wersja. Dwa jednocześnie przesłania dostają przez to różne numery, a unikalny indeks (`document_id`, `version_no`) jest dodatkowym zabezpieczeniem. Wcześniejsze wersje pozostają dostępne do pobrania, każda zaszyfrowana własnym kluczem.

Statusy i historia zmian. Dozwolone zmiany statusu przedstawia rysunek 8. Reguły są zapisane w pakiecie `shared`, z którego korzysta też interfejs, pokazujący tylko dozwolone przyciski. Niedozwolona zmiana daje 409 INVALID_STATUS_TRANSITION z polskim komunikatem, np. „Nie można zmienić statusu z Roboczy na Zaakceptowany”, a żadne pole dokumentu się nie zmienia. Dokument zarchiwizowany jest tylko do odczytu (409 DOCUMENT_ARCHIVED). Każda rzeczywista zmiana tytułu, numeru lub statusu, dodanie wersji, utworzenie i usunięcie dokumentu trafia do historii: kto, kiedy, jakie pole, stara i nowa wartość. Żądanie, które niczego nie zmienia, nie tworzy wpisów.

Lista i usuwanie. Lista zwraca dokumenty użytkownika od ostatnio zmienionego, z filtrami typu i statusu, wyszukiwaniem tekstu w tytule i numerze (bez rozróżniania wielkości liter, także polskich) oraz stronicowaniem (domyślnie 20, maksymalnie 100 pozycji). Usunięcie jest miękkie: dokument dostaje datę usunięcia i znika ze wszystkich endpointów, ale wiersze, historia i zaszyfrowane pliki zostają.

Autoryzacja. Każde zapytanie repozytorium dokumentów zawiera warunek na właściciela i pomija dokumenty usunięte. Cudzy dokument jest nieodróżnialny od nieistniejącego. Każda operacja na nim zwraca 404, przez co nie da się nawet sprawdzić, czy dany identyfikator istnieje. Identyfikatory są losowymi UUID i nie można ich odgadywać po kolei.



Rysunek 8: Diagram stanów dokumentu

Źródło: opracowanie własne

5.2.11 Role i panel biura

Konto ma jedną z dwóch ról: *kierowca* (DRIVER) albo *biuro* (OFFICE). Rejestracja publiczna zawsze tworzy konto kierowcy, dlatego nikt nie może sam nadać sobie uprawnień biura. Konto biura powstaje z konfiguracji: jeśli ustawiono zmienne `OFFICE_USERNAME` i `OFFICE_PASSWORD`, serwer przy starcie zakłada to konto albo, gdy już istnieje, ustawia hasło z konfiguracji. Login i hasło podlegają tym samym regułom co przy rejestracji (sprawdza je walidacja konfiguracji). Jeśli podany login należy już do konta kierowcy, serwer zapisuje błąd w logu i nie zmienia tego konta w konto biura, bo taki login mógł zarejestrować ktokolwiek.

Endpointy są rozdzielone według roli. Pośrednik `requireRole` działa po sprawdzeniu tokenu i przy każdym żądaniu odczytuje rolę z bazy danych (nie z tokenu). Zmiana roli albo usunięcie konta działa dzięki temu od razu. Endpointy `/api/documents` i `/api/profile` są tylko dla kierowców, a `/api/office` tylko dla biura. Konto innej roli dostaje odpowiedź 403 FORBIDDEN.

Maszyna stanów dokumentu (rysunek 8) jest podzielona między role (`ROLE_STATUS_TRANSITIONS` we wspólnym pakiecie). Kierowca przesyła, wycofuje, poprawia i archiwizuje swoje dokumenty. Tylko biuro akceptuje albo odrzuca dokument przesłany. Próba akceptacji przez kierowcę kończy się odpowiedzią 403 `STATUS_CHANGE_NOT_ALLOWED`.

Biuro widzi dokumenty wszystkich kierowców (bez usuniętych) razem z danymi kontaktowymi kierowcy z profilu, może pobierać ich pliki (odszyfrowywane i sprawdzane tak samo jak u

kierowcy) i wydawać decyzje. Nie może natomiast edytować ani usuwać dokumentów. Decyzję zapisuje `POST /api/office/documents/:id/review` z polem `decision` (`ACCEPTED` albo `REJECTED`) i komentarzem, który przy odrzuceniu jest wymagany. Komentarz trafia do historii dokumentu razem z autorem zmiany i kierowca widzi, kto i dlaczego odrzucił dokument. Zmiana statusu jest warunkowa: aktualizacja w bazie ma warunek „status nadal `SUBMITTED`” i odbywa się w transakcji z wpisem historii. Z dwóch jednoczesnych decyzji (np. dwóch pracowników biura) zapisuje się tylko jedna, a druga dostaje `409 INVALID_STATUS_TRANSITION`.

5.3 Baza danych

Dane aplikacji są przechowywane w bazie PostgreSQL 16, a dostęp do nich zapewnia Prisma ORM w wersji 7. Schemat bazy jest zdefiniowany w jednym pliku `backend/prisma/schema.prisma`. Na jego podstawie Prisma generuje klienta z typami TypeScript, dzięki czemu zapytania do nieistniejących kolumn i błędy typów wykrywa już kompilator. Prisma 7 nie korzysta z osobnego silnika zapytań; łączy się z bazą przez sterownik `pg` (`adapter @prisma/adapter-pg`), z limitem 5 sekund na nawiązanie połączenia.

5.3.1 Migracje

Każda zmiana schematu jest zapisywana jako migracja SQL w katalogu `backend/prisma/migrations/` i wersjonowana w repozytorium razem z kodem. Programista tworzy migrację poleceniem `npm run db:migrate -w backend -- --name <nazwa>`. W środowisku Docker migracje stosuje jednorazowa usługa `migrate` (`prisma migrate deploy`), która uruchamia się, gdy baza danych jest gotowa, a backend startuje dopiero po jej poprawnym zakończeniu. Obraz tej usługi powstaje z etapu budowania, bo tylko on zawiera narzędzie wiersza poleceń Prisma. Obraz produkcyjny backendu go nie zawiera.

5.3.2 Model danych

Pierwsza migracja (`init`) tworzy tabelę `users` z kontami użytkowników (tabela 7). Identyfikatory są typu `UUID` i nie da się ich odgadnąć, a to ma znaczenie przy adresach dokumentów w API.

Druga migracja (`driver_profile`) dodaje tabelę `driver_profiles` z danymi kierowcy (tabela 8). Relacja z tabelą `users` jest jeden do jednego (rysunek 9): unikalny klucz obcy `user_id` gwarantuje, że konto ma najwyżej jeden profil. Klauzula `ON DELETE CASCADE` usuwa profil razem z kontem. Wszystkie dane kierowcy są opcjonalne, bo profil powstaje automatycznie przy rejestracji, zanim kierowca go wypełni. Długości kolumn są równe limitom w schemacie walidacji i poprawne dane zawsze mieszczą się w bazie.

Migracja zawiera też ręcznie dopisaną instrukcję, która tworzy pusty profil dla każdego istniejącego już konta:

Tabela 7: Tabela users

Kolumna	Typ	Opis
id	UUID	Klucz główny
username	VARCHAR(30)	Login, unikalny
email	VARCHAR(254)	Adres e-mail, unikalny
password_hash	TEXT	Skrót hasła (argon2), nigdy samo hasło
role	UserRole	Rola: DRIVER (domyślna) albo OFFICE
created_at	TIMESTAMPTZ	Data utworzenia konta
updated_at	TIMESTAMPTZ	Data ostatniej zmiany

Źródło: opracowanie własne

```
INSERT INTO "driver_profiles" ("id", "user_id", "updated_at")
SELECT gen_random_uuid(), "id", CURRENT_TIMESTAMP FROM "users";
```

Dzięki temu zasada „każde konto ma profil” obowiązuje także dla kont założonych przed wprowadzeniem profili, a kod aplikacji nie musi obsługiwać braku profilu.

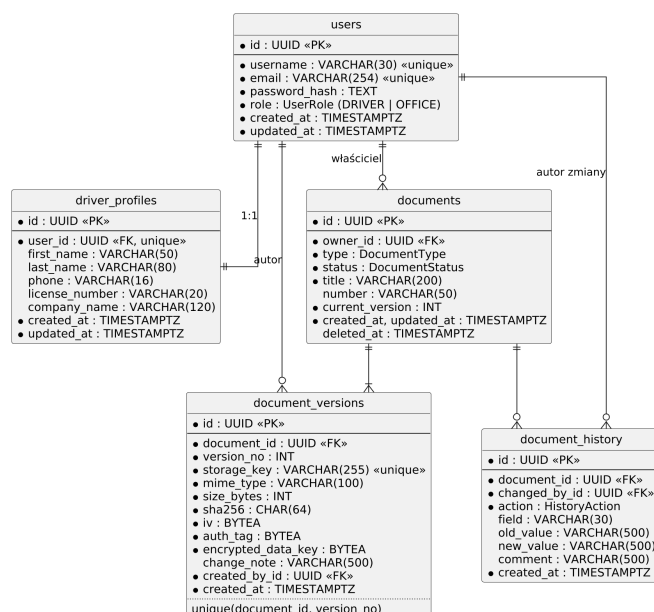
Tabela 8: Tabela driver_profiles

Kolumna	Typ	Opis
id	UUID	Klucz główny
user_id	UUID	Właściciel, klucz obcy do users, unikalny
first_name	VARCHAR(50)	Imię
last_name	VARCHAR(80)	Nazwisko
phone	VARCHAR(16)	Telefon w postaci znormalizowanej, np. +48601234567
license_number	VARCHAR(20)	Numer prawa jazdy
company_name	VARCHAR(120)	Firma przewozowa
created_at, updated_at	TIMESTAMPTZ	Data utworzenia i ostatniej zmiany

Źródło: opracowanie własne

Trzecia migracja (documents) dodaje tabele dokumentów, ich wersji i historii zmian oraz typy wyliczeniowe DocumentType, DocumentStatus i HistoryAction (rysunek 9):

- **documents**: metadane dokumentu, numer bieżącej wersji i data usunięcia (usuwanie miękkie). Indeks (owner_id, deleted_at, updated_at) odpowiada zapytaniu listy: dokumenty jednego właściciela, bez usuniętych, od ostatnio zmienionego.
- **document_versions**: jedna wersja pliku z kluczem obiektu w MinIO, formatem, rozmiarem, skrótem SHA-256 oryginału i parametrami szyfrowania: wektor inicjujący (12 B), znacznik GCM (16 B) oraz zaszyfrowany klucz danych (60 B) w kolumnach typu BYTEA. Para (document_id, version_no) jest unikalna.
- **document_history**: dziennik zmian z autorem, rodzajem zmiany, polem oraz starą i nową wartością.



Rysunek 9: Diagram ERD bazy danych

Źródło: opracowanie własne

Czwarta migracja (`office_role`) dodaje typ wyliczeniowy `UserRole`, kolumnę `role` w tabeli `users` z wartością domyślną `DRIVER` (istniejące konta stały się kontami kierowców bez przepisywania danych) oraz kolumnę `comment` w tabeli `document_history` na komentarz biura do decyzji (podrozdział 5.2.11).

Samych plików, nawet zaszyfrowanych, nie ma w bazie danych. Przechowuje je MinIO, co ogranicza rozmiar bazy i jej kopii zapasowych. Z kolei w MinIO nie ma niczego, co pozwalałoby odszyfrować pliki. Do odczytu dokumentu potrzebny jest dostęp do obu magazynów oraz klucz główny z konfiguracji serwera.

5.4 Docker

Cały system jest opisany w pliku `docker-compose.yml` i uruchamiany poleceniem `docker compose up -d --build --wait` (w repozytorium: `npm run docker:up`). Opcja `--wait` kończy polecenie dopiero wtedy, gdy wszystkie kontenery zgłoszą stan *healthy*. Usługi zestawiono w tabeli 9.

5.4.1 Wieloetapowe obrazy

Backend i frontend mają własne pliki `Dockerfile` zbudowane w dwóch etapach. Etap *build* instaluje wszystkie zależności (także deweloperskie, np. kompilator TypeScript) i kompiluje kod. Etap *runtime* zawiera tylko to, co jest potrzebne do działania: w przypadku backendu są to skompilowany kod JavaScript i zależności produkcyjne, a w przypadku frontendu statyczne pliki aplikacji serwowane przez nginx. Kontekstem budowania jest katalog główny repozytorium, ponieważ oba pakiety korzystają ze wspólnego pakietu `shared`. Polecenie `npm ci -w` instaluje

Tabela 9: Usługi zdefiniowane w pliku `docker-compose.yml`

Usługa	Obraz	Port hosta	Healthcheck
frontend	własny (nginx)	8090	GET /nginx-health
backend	własny (Node.js 22)	–	GET /health
migrate	własny (etap <i>migrate</i>)	–	zadanie jednorazowe
postgres	postgres:16-alpine	5432*	pg_isready
minio	chainguard/minio	9000, 9001*	mc ready
minio-init	chainguard/minio	–	zadanie jednorazowe
prometheus	prom/prometheus	9090*	GET /-/healthy
grafana	grafana/grafana	3001	GET /api/health
* port dostępny tylko z komputera, na którym działa Docker (127.0.0.1)			

Źródło: opracowanie własne

przy tym zależności tylko potrzebnych pakietów monorepo. Gotowy obraz backendu zajmuje około 250 MB, a obraz frontendu około 83 MB.

Plik `backend/Dockerfile` ma jeszcze trzeci etap, *migrate*. Powstaje on z etapu *build*, który zawiera narzędzie wiersza poleceń Prisma, i służy jednorazowej usłudze *migrate* do stosowania migracji bazy danych (podrozdział 5.3). Narzędzie to jest zależnością deweloperską i nie trafia do obrazu produkcyjnego.

Oba obrazy działają na koncie użytkownika bez uprawnień administratora: backend jako użytkownik `node` (UID 1000), a frontend na obrazie `nginx-unprivileged` jako użytkownik `nginx` (UID 101), nasłuchując na nieuprzywilejowanym porcie 8080. Backend jest uruchamiany z opcją `init: true`, która jako proces numer 1 umieszcza w kontenerze program `tini`. Przekazuje on procesowi Node.js sygnał `SIGTERM`, dzięki czemu `docker compose stop` kończy kontener od razu, bez czekania na 10-sekundowy limit.

5.4.2 Serwer nginx

Konfiguracja `nginx` (`frontend/nginx.conf`) realizuje trzy zadania. Nieznane ścieżki kieruje do `index.html`, bo nawigacją zarządza router aplikacji React. Żądania `/api/*` przekazuje do backendu. Dodaje też nagłówki bezpieczeństwa (`X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`). Adres backendu jest rozwiązywany przez wbudowany serwer DNS Dockera przy każdym żądaniu (dyrektywa `resolver` i zmienna w `proxy_pass`). Po restarcie kontenera backendu, który może dostać nowy adres IP, `nginx` od razu kieruje ruch pod właściwy adres; sprawdzono to, restartując backend przy działającym systemie. Pliki z katalogu `/assets/` mają w nazwie skrót zawartości, dlatego przeglądarka może je przechowywać przez rok, natomiast `index.html` nie jest zapisywany w pamięci podręcznej.

5.4.3 Kolejność uruchamiania i odporność na awarie

Każda usługa ma zdefiniowany `healthcheck` oraz politykę `restart: unless-stopped`. Kolejność uruchamiania wynika z warunków `depends_on`: backend startuje dopiero wtedy, gdy PostgreSQL i MinIO są *healthy*, a zadania `minio-init` i `migrate` zakończyły się sukcesem (`service_completed_successfully`). Pierwsze zadanie tworzy bucket na dokumenty z wyłączonym dostępem anonimowym, drugie stosuje brakujące migracje bazy danych. Oba można bezpiecznie uruchamiać przy każdym starcie: opcja `--ignore-existing` pomija istniejący bucket, a `prisma migrate deploy` pomija zastosowane już migracje. Frontend czeka na zdrowy backend, a Grafana na Prometheusa. Fragment definicji backendu:

```
backend:
  restart: unless-stopped
  init: true
  depends_on:
    postgres: { condition: service_healthy }
    minio:     { condition: service_healthy }
    minio-init: { condition: service_completed_successfully }
    migrate:   { condition: service_completed_successfully }
  healthcheck:
    test: ['CMD', 'wget', '-q', '-O', '/dev/null',
          'http://127.0.0.1:3000/health']
    interval: 10s
    timeout: 5s
    retries: 3
    start_period: 15s
```

5.4.4 Sieci i wolumeny

Podział na sieci `app`, `data` i `monitoring` opisano w rozdziale 4. Dane trwale znajdują się w nazwanych wolumenach: `pgdata` (baza danych), `miniodata` (pliki dokumentów), `promdata` (szeregi czasowe metryk) i `grafanadata`. Polecenie `docker compose down` usuwa kontenery, ale zachowuje wolumeny. Porty bazy danych, MinIO i Prometheusa są wystawione tylko na adresie `127.0.0.1` i są dostępne wyłącznie z komputera, na którym działa Docker. Służą programiście do uruchamiania backendu poza kontenerem (`npm run dev`).

5.4.5 Konfiguracja i sekrety

Wszystkie parametry są przekazywane przez zmienne środowiskowe z pliku `.env`, który nie trafia do repozytorium. W repozytorium znajduje się wzorzec `.env.example`, a skrypt `npm run env:init` tworzy na jego podstawie plik `.env` z losowymi hasłami do bazy, MinIO i Grafany, sekretem JWT oraz 256-bitowym kluczem głównym szyfrowania. Zmienne wymagane są

zapisane w postaci `${JWT_SECRET:?komunikat}`, dzięki czemu `docker compose` odmawia uruchomienia systemu, gdy któregoś sekretu brakuje, zamiast uruchomić go z pustym hasłem. Zmienne `OFFICE_USERNAME` i `OFFICE_PASSWORD` określają konto biura zakładane przy starcie backendu (podrozdział 5.2.11). Hasło biura również generuje `npm run env:init`. Puste wartości oznaczają system bez konta biura.

5.4.6 Środowisko testowe

Plik `docker-compose.test.yml` definiuje osobny projekt (`driver-docs-test`) z bazą PostgreSQL i magazynem MinIO na potrzeby testów funkcjonalnych API. Dane tych kontenerów są przechowywane w pamięci (`tmpfs`). Każde uruchomienie zaczyna się od pustego stanu, a testy nie mogą uszkodzić danych środowiska deweloperskiego. Kontenery testowe używają innych portów (5433 i 9100) i mogą działać jednocześnie z głównym systemem.

5.4.7 Napotkany problem: obrazy MinIO

W 2025 roku firma MinIO przestała publikować obrazy wersji Community w serwisie Docker Hub. W czasie realizacji projektu repozytorium `minio/minio` nie było tam już dostępne i próba pobrania obrazu kończyła się błędem. Projekt korzystał najpierw z rejestru `quay.io`, ale i tam obrazy przestały być dostępne bez logowania. Zauważyłem to dopiero przy pierwszym przebiegu ciągłej integracji na GitHubie, bo na moim komputerze obrazy były już pobrane. Projekt używa dlatego darmowego obrazu MinIO przygotowanego przez firmę Chainguard (`cgr.dev/chainguard/minio`), przypiętego skrótem SHA-256. Obraz działa jako użytkownik bez uprawnień administratora i zawiera klienta `mc`, dlatego służy też do zadania `minio-init` i do kontroli stanu (`mc ready`). Wniosek z tej sytuacji: wersje obrazów należy przypinać, a pobieranie obrazów sprawdzać na czystym środowisku, np. w potoku CI. Obrazy MinIO, Prometheusa i Grafany mają przypiętą dokładną wersję, a obrazy bazowe (`node:22`, `postgres:16`) wersję główną, która nadal otrzymuje poprawki bezpieczeństwa.

5.5 Testy

5.5.1 Strategia

Podstawą są testy funkcjonalne. Sprawdzają one aplikację tak, jak używa jej klient: żądania HTTP trafiają do działającego API połączonego z prawdziwą bazą PostgreSQL i prawdziwym magazynem MinIO, a scenariusze interfejsu wykonuje przeglądarka. Test ocenia tylko to, co widać z zewnątrz, czyli kod odpowiedzi, jej treść i nagłówki albo zawartość ekranu. Testy jednostkowe uzupełniają je tam, gdzie pojedyncza funkcja ma wiele przypadków brzegowych, na przykład w walidacji danych, obliczeniach kompresji albo regułach zmiany statusu.

Błędy naprawiano zawsze w tej samej kolejności. Najpierw ustalano przyczynę, potem powstawał test odtwarzający błąd, który przed poprawką nie przechodził, a dopiero na końcu

zmieniano kod. Test zostaje w zestawie i pilnuje, żeby błąd nie wrócił. Podobnie powstawała większość nowych funkcji: testy funkcjonalne pisano przed kodem i sprawdzano, że kończą się błędem z oczekiwanego powodu, np. z powodu braku endpointu.

5.5.2 Środowisko testowe

Testy backendu uruchamia Vitest, a żądania HTTP wysyła biblioteka Supertest, bez otwierania portu sieciowego. Baza danych i MinIO do testów działają w osobnych kontenerach z pliku `docker-compose.test.yml`, z danymi w pamięci. Przed testami stosowane są wszystkie migracje, a przed każdym testem tabele są czyszczone jednym poleceniem `TRUNCATE`, dzięki czemu testy nie wpływają na siebie nawzajem. Funkcje pomocnicze budują aplikację tak samo jak `server.ts`, ale pozwalają podmienić wybrane ustawienia. W ten sposób sprawdzane są też awarie zależności: wystarczy wskazać adres bazy albo MinIO, pod którym nic nie działa.

Scenariusze interfejsu wykonuje Playwright w przeglądarce Chromium z emulacją telefonu Pixel 7 (rozmiar ekranu, obsługa dotyku, agent użytkownika). Scenariusze panelu biura przełączają przeglądarkę na ekran komputera 1440×900. Playwright sam uruchamia backend na bazie testowej oraz zbudowaną wersję frontendu (`vite preview`). Aparat zastępuje wirtualna kamera Chromium z automatyczną zgodą na dostęp, włączana flagami:

```
--use-fake-device-for-media-stream
--use-fake-ui-for-media-stream
```

Elementy strony test wyszukiwań po etykietach i rolach (`getByLabel`, `getByRole`), tak jak odnajduje je osoba korzystająca z czytnika ekranu. Przy okazji sprawdza to, czy pola formularzy są poprawnie opisane.

5.5.3 Zakres testów

Tabela 10 zestawia, co sprawdzają testy w poszczególnych obszarach. Liczby testów i pokrycie kodu podaje tabela 12.

Aby sprawdzić, czy testy rzeczywiście wykrywają błędy, kilka razy celowo psuto kod. Z zapytania o dokument usuwano warunek na właściciela, plik zapisywano w MinIO bez szyfrowania, pomijano reguły zmiany statusu, a błąd 500 ujawniał treść wyjątku. Każdą z takich usterek wykrył co najmniej jeden test.

5.5.4 Błędy wykryte przez testy

Testy funkcjonalne wykryły kilka błędów, których nie ujawniłyby testy jednostkowe. Najciekawszy z nich dotyczył sesji. Po odświeżeniu strony dokumentu aplikacja pokazywała komunikat „Sesja wygasła”, mimo że użytkownik był zalogowany. Zapis przebiegu testu pokazał, że pierwsze zapytanie po wczytaniu strony dostaje odpowiedź 401. Komponent `AuthProvider`

Tabela 10: Zakres testów

Obszar	Sprawdzane zachowanie
Stan i konfiguracja	/health i /health/ready (503 przy awarii bazy, MinIO lub braku bucketa, bez szczegółów błędu); serwer z błędną konfiguracją kończy pracę z kodem 1 i wymienia wszystkie błędy bez wartości sekretów; jednolity format błędów (400, 404, 413)
Rejestracja i logowanie	walidacja pól i wartości graniczne; zajęty login lub e-mail (409); ten sam komunikat dla złego loginu i złego hasła; limit nieudanych prób (429); odrzucanie tokenów wygasłych, podpisanych innym kluczem, niepodpisanych i z obcym wystawcą
Profil	normalizacja danych, polskie znaki, oddzielne profile użytkowników
Dokumenty i pliki	pobrany plik identyczny z przesłanym; w MinIO szyfrogram bez czytelnej treści; format rozpoznawany po zawartości; limit 10 MB; kolejne wersje; reguły zmiany statusu; usuwanie miękkie; 404 dla cudzego dokumentu; odmowa wydania pliku zmienionego w magazynie
Role i panel biura	403 dla konta innej roli; dokumenty wszystkich kierowców z filtrami; akceptacja i odrzucenie z wymaganym powodem; z dwóch jednoczesnych decyzji zapisuje się jedna; konto biura z konfiguracji nie przejmuje istniejącego konta kierowcy
Interfejs	rejestracja, logowanie, sesja po odświeżeniu strony, praca z dokumentami, profil; pełna ścieżka kierowcy od rejestracji do wylogowania; obieg odrzucenie, poprawa, akceptacja w dwóch przeglądarkach naraz
Aparat	zdjęcie z obrotem i kompresją do pliku JPEG poniżej 1 MB; wyłączenie kamery po zamknięciu widoku; odmowa dostępu i brak aparatu (pole zapasowe); zmniejszenie zdjęcia 4000×3000 z galerii
PWA	manifest i ikony; start bez sieci z banerem braku połączenia; brak odpowiedzi API w pamięci service workera
Monitoring	metryki z etykietą szablonu trasy; liczniki nieudanych logowań i przesłanych plików; 503 z /health po zablokowaniu pętli zdarzeń; poprawność reguł alertów (promtool, amtool)

Źródło: opracowanie własne

przekazywał token do klienta API w efekcie (`useEffect`), a React wykonuje efekty komponentów potomnych przed efektem rodzica. Zapytania o dane wychodziły zatem, zanim klient dostał token. Przy zwykłym logowaniu błąd nie występował, bo token był ustawiany od razu. Test odtwarzający błąd (odświeżenie strony, a potem otwarcie dokumentu z adresu) nie przechodził. Po przeniesieniu tokenu do osobnego modułu poza Reactem, odczytywanego przez klienta API przy każdym żądaniu, test przeszedł. Pozostałe błędy zestawia tabela 11.

Tabela 11: Pozostałe błędy wykryte przez testy

Objaw	Przyczyna i poprawka
Kamera wyłączała się zaraz po włączeniu (użytkownik widziałby zamrożony obraz)	efekt w hooku <code>useCamera</code> zależał od stanu aparatu, a zmiana stanu uruchamiała sprzątanie; efekt działa teraz przez cały czas wyświetlania widoku
Po wylogowaniu następny użytkownik trafiał na stronę poprzedniego	strażnik tras zapamiętywał adres powrotu także przy wylogowaniu przyciskiem; adres jest zapamiętywany tylko po wygaśnięciu sesji
Metryki HTTP miały zły szablon trasy	Express przywraca ścieżkę routera przed końcem odpowiedzi; ścieżka jest zapamiętywana wcześniej
Zawieszenie procesu mogło pozostać niezauważone	funkcja <code>monitorEventLoopDelay</code> pomijała blokadę tuż po wyzerowaniu pomiaru; zastąpiono ją własnym pomiarem
Biuro widziało stary status dokumentu ponownie przesłanego przez kierowcę	dane z pamięci podręcznej były uznawane za aktualne przez 30 s; listy i szczegóły dokumentów są pobierane przy każdym wejściu na ekran
Status wybrany zaraz po wpisaniu wyszukiwanego tekstu był cofany	opóźnione o 300 ms wyszukiwanie korzystało ze starego stanu filtrów, bo React Router renderuje nawigację asynchronicznie; poprawka buduje adres z bieżącego URL i zatwierdza zmianę synchronicznie (<code>flushSync</code>), a test celowo blokuje wątek przeglądarki, żeby oba zdarzenia zawsze nastąpiły w tej samej chwili

Źródło: opracowanie własne

5.5.5 Stabilność testów w przeglądarce

Pierwsze przebiegi testów interfejsu kończyły się przekroczeniem czasu w losowych scenariuszach. Zapisy przebiegu (*trace*) i pomiary wykazały, że nie był to błąd aplikacji. Kilka przeglądarek naraz konkurowało o procesor z haszowaniem haseł `argon2`, które jest celowo kosztowne, i z dekodowaniem zdjęć 12 Mpx. Dodatkowo serwer deweloperski Vite kompilował zależności przy pierwszym wejściu. Testy korzystają teraz ze zbudowanej aplikacji, scenariusze wykonują się po kolei, limity czasu są dłuższe, a nieudany test jest raz powtarzany. Test, który przechodzi dopiero za drugim razem, Playwright oznacza jako niestabilny (*flaky*), dzięki czemu problem nie znika z raportu. Parametrów `argon2` nie osłabiono, bo testy sprawdzałyby wtedy inną konfigurację niż produkcyjna. Cały zestaw 36 scenariuszy trwa od dwóch do pięciu minut.

5.5.6 Testy obrazów Docker

Te same scenariusze można uruchomić na obrazach Docker, czyli na konfiguracji, która jest wdrażana (nginx, backend, baza, MinIO). Początkowo testy uruchamiano na działającym stosie aplikacji, co zapisywało konta i dokumenty testowe w jego bazie obok prawdziwych danych. Po zauważeniu problemu dane testowe usunięto, a testy obrazów uruchamia skrypt `scripts/e2e-docker.mjs` (`npm run test:e2e:docker`). Tworzy on jednorazową kopię stosu: osobny projekt Docker Compose z pustymi wolumenami i innymi portami. Po udanych testach kopia jest usuwana razem z danymi. Po nieudanych zostaje do analizy, a logi kontenerów są zapisywane do pliku. Konfiguracja Playwright odmawia pracy z zewnętrznym adresem uruchomionym inaczej niż przez ten skrypt. Na kopii stosu przechodzi cały zestaw 36 scenariuszy.

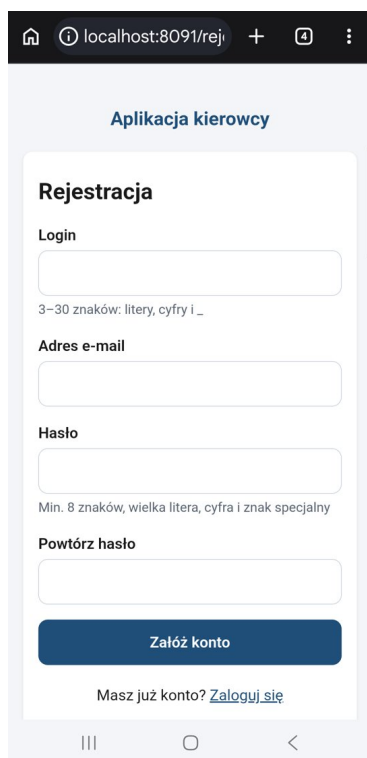
5.5.7 Test na prawdziwym telefonie

Aparat, obsługę dotykem i instalację PWA sprawdzono ręcznie na telefonie Samsung Galaxy A26 z systemem Android 16 i przeglądarką Chrome, podłączonym przez USB. Polecenie `adb reverse` przekierowuje port aplikacji, dzięki czemu telefon otwiera ją pod adresem `localhost`, który przeglądarka uznaje za bezpieczny kontekst. Sprawdzono rejestrację, dodanie dokumentu CMR, zgodę na dostęp do aparatu, zdjęcie tylnym aparatem, kadrowanie, zapis, przesłanie do biura, decyzję biura widoczną na telefonie oraz instalację aplikacji na ekranie głównym (rysunki 4, 5 i 10). Aparat przekazuje klatki 1440×1080 pikseli. Po wykadrowaniu do 1080×810 pikseli i kompresji zdjęcie zajęło 70,1 kB zamiast 367,8 kB. Test potwierdził wyniki uzyskane w emulacji i nie wykazał problemów z uprawnieniami aparatu, dotykem ani instalacją.

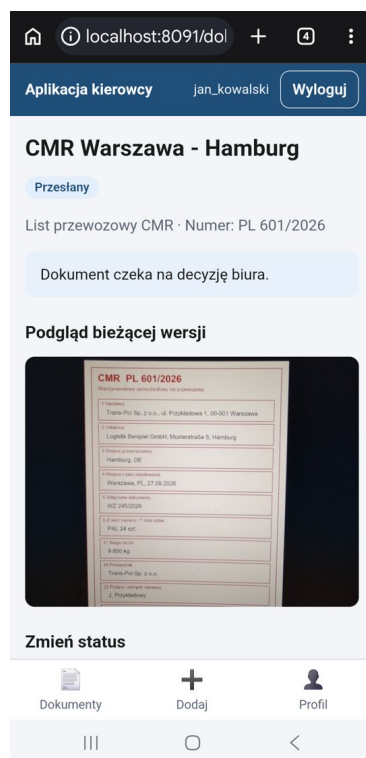
5.5.8 Pokrycie kodu

Pokrycie mierzy Vitest z modułem V8 (`npm run test:coverage`). Interfejs jest sprawdzany głównie w przeglądarce, dlatego dla frontendu mierzone jest także pokrycie kodu przez testy Playwright (`npm run test:e2e:coverage`). Chromium zbiera pokrycie każdej strony, a biblioteka `monocart-coverage-reports` przypisuje je do plików źródłowych przez mapy źródeł. Mapy powstają tylko na potrzeby tego pomiaru i nie trafiają do obrazu produkcyjnego. Wyniki zawiera tabela 12.

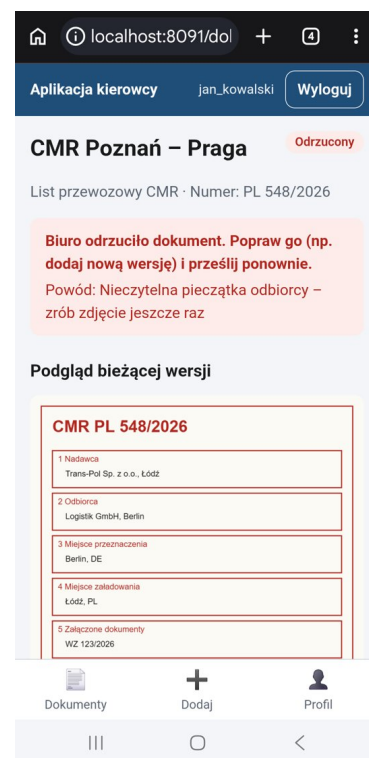
Niskie pokrycie frontendu samymi testami jednostkowymi jest zamierzone. Testy jednostkowe dotyczą czystej logiki (obliczenia kompresji, kontrola pliku, formatowanie, opis historii), a komponenty są sprawdzane w przeglądarce, tak jak używa ich kierowca. Z pomiaru backendu wyłączono wygenerowany kod klienta Prisma oraz plik startowy `server.ts`, który sprawdza osobny test uruchamiający prawdziwy proces serwera.



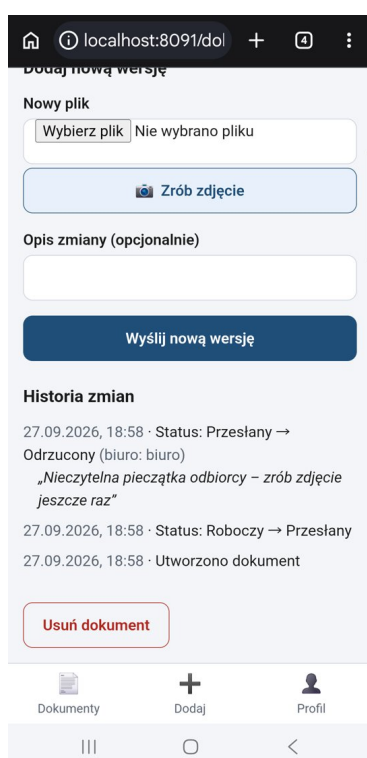
(a) rejestracja



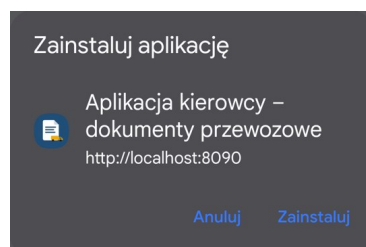
(b) dokument przesłany do biura



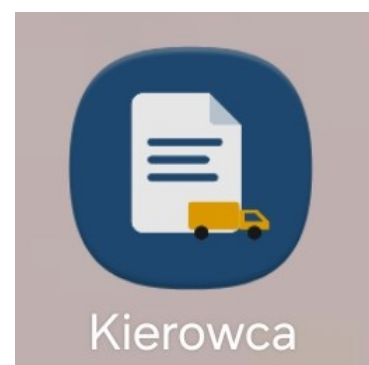
(c) dokument odrzucony przez biuro



(d) historia z komentarzem biura



(e) instalacja PWA



(f) ikona na ekranie głównym

Rysunek 10: Test na prawdziwym telefonie (Samsung Galaxy A26, Android 16, Chrome)

Źródło: opracowanie własne (zrzuty ekranu telefonu)

Tabela 12: Pokrycie kodu testami

Pakiet (rodzaj testów)	Instrukcje	Gałęzie	Funkcje	Linie
Backend (funkcjonalne API i jednostkowe, 189 testów)	95,1 %	85,9 %	97,5 %	97,0 %
Pakiet shared (jednostkowe, 109 testów)	96,8 %	90,3 %	100 %	96,8 %
Frontend (Playwright, 36 scenariuszy)	94,1 %	81,9 %	89,9 %	92,0 %
Frontend (tylko jednostkowe, 65 testów)	13,8 %	15,4 %	10,4 %	13,8 %

Źródło: opracowanie własne

5.5.9 Wyniki uruchomienia testów

Rysunki 11–13 pokazują wyniki ostatniego uruchomienia wszystkich zestawów testów w terminalu, na infrastrukturze testowej (`npm run test:infra:up`). Wszystkie testy kończą się powodzeniem: 189 testów backendu, 109 testów pakietu shared, 65 testów jednostkowych frontendu i 36 scenariuszy E2E. Ostrzeżenie `DeprecationWarning` widoczne w wynikach pochodzi z biblioteki `@prisma/adapters-pg` (kilka zapytań w jednej transakcji) i nie jest błędem projektu. Zniknie po aktualizacji Prismy, której wersja jest celowo przypięta.

```

Terminal – testy backendu

PS C:\repo\JEE_Projekt_PR_122217> npm test -w backend -- --reporter=default --reporter.summary=false --slowTestThreshold=600000
RUN v5.0.1 C:/repo/JEE_Projekt_PR_122217/backend

  ✓ test/documents.test.ts (50 tests) 20364ms
(node:15172) DeprecationWarning: Calling client.query() when the client is already executing a query is deprecated and will be
removed in pg@9.0. Use async/await or an external async flow control mechanism instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
  ✓ test/office.test.ts (24 tests) 13258ms
  ✓ test/auth.test.ts (25 tests) 9699ms
  ✓ test/server-startup.test.ts (2 tests) 6942ms
  ✓ test/profile.test.ts (12 tests) 3898ms
  ✓ test/metrics.test.ts (8 tests) 3274ms
  ✓ test/health.test.ts (8 tests) 1186ms
  ✓ test/errors.test.ts (7 tests) 644ms
  ✓ test/test-database.test.ts (2 tests) 516ms
  ✓ test/docs.test.ts (3 tests) 566ms
  ✓ src/middleware/error-handler.test.ts (6 tests) 162ms
  ✓ src/services/encryption.service.test.ts (14 tests) 67ms
  ✓ src/config/env.test.ts (19 tests) 45ms
  ✓ src/lib/file-type.test.ts (9 tests) 10ms

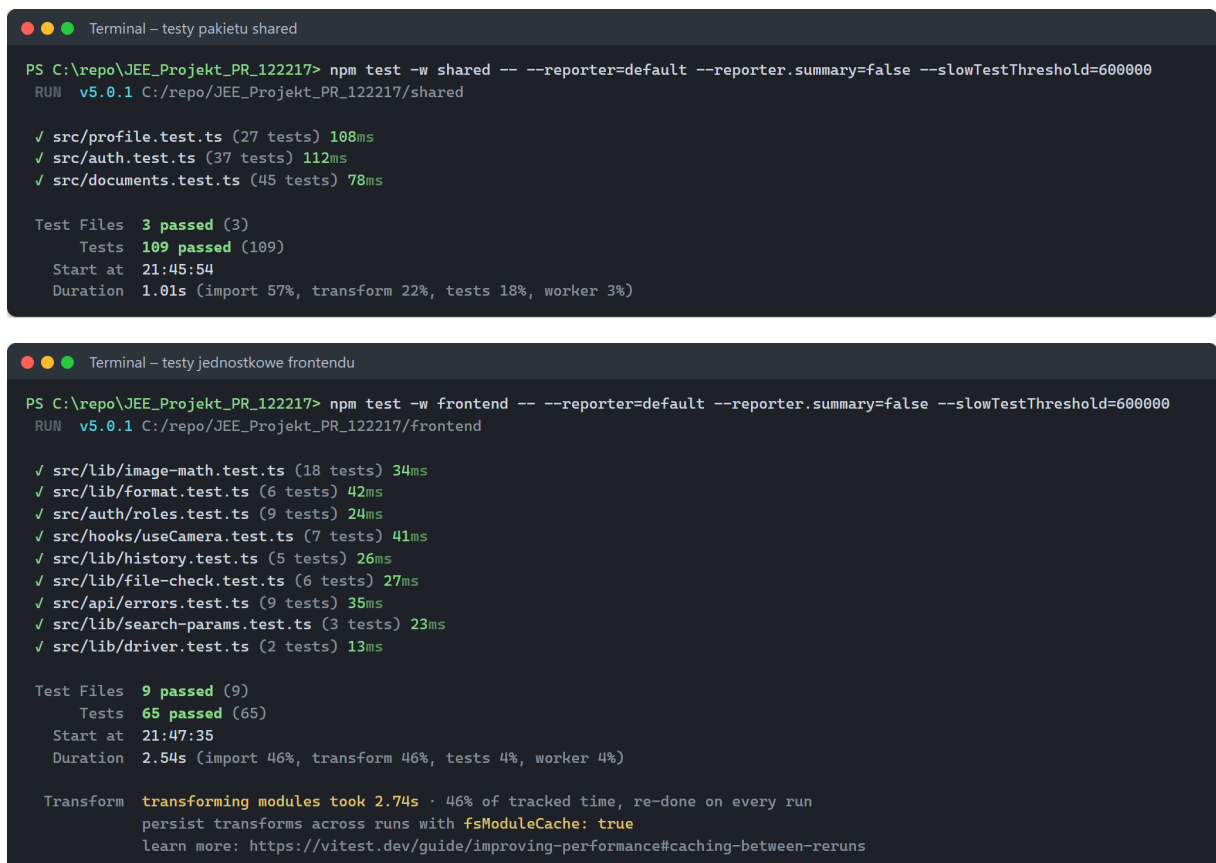
Test Files 14 passed (14)
Tests 189 passed (189)
Start at 21:46:01
Duration 89.36s (tests 78%, import 20%, transform 2%)

Isolate 14 workers spawned · ~387ms startup each (spawn + environment, per file)
at least ~5.03s faster with isolate: false - reuses workers across files instead of one per file

```

Rysunek 11: Testy backendu (funkcjonalne API i jednostkowe)

Źródło: opracowanie własne (zrzut terminala)



The image contains two terminal screenshots. The top terminal, titled 'Terminal - testy pakietu shared', shows the command `npm test -w shared -- --reporter=default --reporter.summary=false --slowTestThreshold=600000` being executed. It reports that 3 test files passed, totaling 109 tests. The bottom terminal, titled 'Terminal - testy jednostkowe frontendu', shows the command `npm test -w frontend -- --reporter=default --reporter.summary=false --slowTestThreshold=600000`. It reports that 9 test files passed, totaling 65 tests. Both terminals also display performance metrics like duration and transform time.

```
Terminal - testy pakietu shared

PS C:\repo\JEE_Projekt_PR_122217> npm test -w shared -- --reporter=default --reporter.summary=false --slowTestThreshold=600000
RUN v5.0.1 C:/repo/JEE_Projekt_PR_122217/shared

✓ src/profile.test.ts (27 tests) 108ms
✓ src/auth.test.ts (37 tests) 112ms
✓ src/documents.test.ts (45 tests) 78ms

Test Files 3 passed (3)
Tests 109 passed (109)
Start at 21:45:54
Duration 1.01s (import 57%, transform 22%, tests 18%, worker 3%)

Terminal - testy jednostkowe frontendu

PS C:\repo\JEE_Projekt_PR_122217> npm test -w frontend -- --reporter=default --reporter.summary=false --slowTestThreshold=600000
RUN v5.0.1 C:/repo/JEE_Projekt_PR_122217/frontend

✓ src/lib/image-math.test.ts (18 tests) 34ms
✓ src/lib/format.test.ts (6 tests) 42ms
✓ src/auth/roles.test.ts (9 tests) 24ms
✓ src/hooks/useCamera.test.ts (7 tests) 41ms
✓ src/lib/history.test.ts (5 tests) 26ms
✓ src/lib/file-check.test.ts (6 tests) 27ms
✓ src/api/errors.test.ts (9 tests) 35ms
✓ src/lib/search-params.test.ts (3 tests) 23ms
✓ src/lib/driver.test.ts (2 tests) 13ms

Test Files 9 passed (9)
Tests 65 passed (65)
Start at 21:47:35
Duration 2.54s (import 46%, transform 46%, tests 4%, worker 4%)

Transform transforming modules took 2.74s · 46% of tracked time, re-done on every run
persist transforms across runs with fsModuleCache: true
learn more: https://vitest.dev/guide/improving-performance#cache-between-reruns
```

Rysunek 12: Testy jednostkowe pakietu shared i frontendu

Źródło: opracowanie własne (zrzut terminala)

```
Terminal - testy E2E (Playwright)

PS C:\repo\JEE_Projekt_PR_122217> npx playwright test --reporter=list
Running 36 tests using 1 worker

ok 1 [mobile-chromium] › tests\auth.spec.ts:11:3 › registration › registers, logs in automatically and shows the start page (5.5s)
ok 2 [mobile-chromium] › tests\auth.spec.ts:20:3 › registration › shows validation errors next to the fields and sends nothing (2.8s)
ok 3 [mobile-chromium] › tests\auth.spec.ts:41:3 › registration › shows the API error next to the username when it is taken (2.5s)
ok 4 [mobile-chromium] › tests\auth.spec.ts:58:3 › login and logout › logs in with correct credentials (6.0s)
ok 5 [mobile-chromium] › tests\auth.spec.ts:67:3 › login and logout › shows a message for a wrong password and stays on the login page (4.2s)
ok 6 [mobile-chromium] › tests\auth.spec.ts:77:3 › login and logout › logs out and protects the pages again (4.7s)
ok 7 [mobile-chromium] › tests\auth.spec.ts:87:3 › login and logout › after an explicit logout, the next login starts at the document list (4.4s)
ok 8 [mobile-chromium] › tests\auth.spec.ts:99:3 › login and logout › keeps the session after reloading the page (3.0s)
ok 9 [mobile-chromium] › tests\auth.spec.ts:107:3 › login and logout › sends a logged-out user to the login page and back to the requested page (2.7s)
ok 10 [mobile-chromium] › tests\auth.spec.ts:119:3 › login and logout › logs out with a message when the stored session is no longer valid (3.8s)
ok 11 [mobile-chromium] › tests\auth.spec.ts:135:3 › mobile layout › shows the main navigation as a bottom bar with large touch targets (2.7s)
ok 12 [mobile-chromium] › tests\camera.spec.ts:61:1 › takes a photo with the camera, crops and rotates it, and saves it as a document (5.0s)
ok 13 [mobile-chromium] › tests\camera.spec.ts:92:1 › stops the camera when the camera view is closed (3.8s)
ok 14 [mobile-chromium] › tests\camera.spec.ts:117:1 › explains a refused camera permission and offers the file fallback (3.2s)
ok 15 [mobile-chromium] › tests\camera.spec.ts:137:1 › falls back to the file input when the browser has no camera API (3.2s)
ok 16 [mobile-chromium] › tests\camera.spec.ts:149:1 › compresses a large photo chosen from the device before uploading it (7.2s)
ok 17 [mobile-chromium] › tests\documents.spec.ts:36:1 › adds a document from a file and shows it in the list and in the preview (3.9s)
ok 18 [mobile-chromium] › tests\documents.spec.ts:58:1 › loads the documents right after a page reload (token restored before the first request) (3.8s)
ok 19 [mobile-chromium] › tests\documents.spec.ts:71:1 › shows the server rejection of a disguised file next to the file field (3.0s)
ok 20 [mobile-chromium] › tests\documents.spec.ts:80:1 › filters the list by type and status and searches by text (5.0s)
ok 21 [mobile-chromium] › tests\documents.spec.ts:101:1 › keeps a filter chosen while the search text is still being applied (3.5s)
ok 22 [mobile-chromium] › tests\documents.spec.ts:115:1 › adds a new version: two versions and the history entry are shown (3.2s)
ok 23 [mobile-chromium] › tests\documents.spec.ts:131:1 › changes the status; the new status is visible in the list (4.1s)
ok 24 [mobile-chromium] › tests\documents.spec.ts:147:1 › edits the title and number (3.7s)
ok 25 [mobile-chromium] › tests\documents.spec.ts:161:1 › downloads a version with the original content (3.8s)
ok 26 [mobile-chromium] › tests\documents.spec.ts:178:1 › deletes a document after confirmation (3.5s)
ok 27 [mobile-chromium] › tests\documents.spec.ts:189:3 › profile › saves the profile; values are normalized and survive a reload (3.0s)
ok 28 [mobile-chromium] › tests\documents.spec.ts:205:3 › profile › shows validation errors next to the fields (2.8s)
ok 29 [mobile-chromium] › tests\full-scenario.spec.ts:11:1 › full driver scenario: registration to logout (7.5s)
ok 30 [mobile-chromium] › tests\office.spec.ts:85:3 › office panel in a desktop browser › the office sees submitted documents of all drivers and filters by driver (5.2s)
ok 31 [mobile-chromium] › tests\office.spec.ts:107:3 › office panel in a desktop browser › keeps a filter chosen while the search text is still being applied (2.9s)
[WebServer] (node:36972) DeprecationWarning: Calling client.query() when the client is already executing a query is deprecated and will be removed in pg@9.0. Use async/await or an external async flow control mechanism instead.
[WebServer] (Use 'node --trace-deprecation ...' to show where the warning was created)
ok 32 [mobile-chromium] › tests\office.spec.ts:118:3 › office panel in a desktop browser › reject with a reason → the driver corrects and resubmits → the office accepts (27.8s)
ok 33 [mobile-chromium] › tests\office.spec.ts:172:1 › keeps drivers and the office in their own parts of the application (8.5s)
ok 34 [mobile-chromium] › tests\pwa.spec.ts:21:1 › links a valid web app manifest with installable icons (2.9s)
ok 35 [mobile-chromium] › tests\pwa.spec.ts:49:1 › starts offline from the service worker cache and shows the offline banner (5.4s)
ok 36 [mobile-chromium] › tests\pwa.spec.ts:66:1 › never stores API responses or document files in the service worker cache (6.8s)

36 passed (4.1m)
```

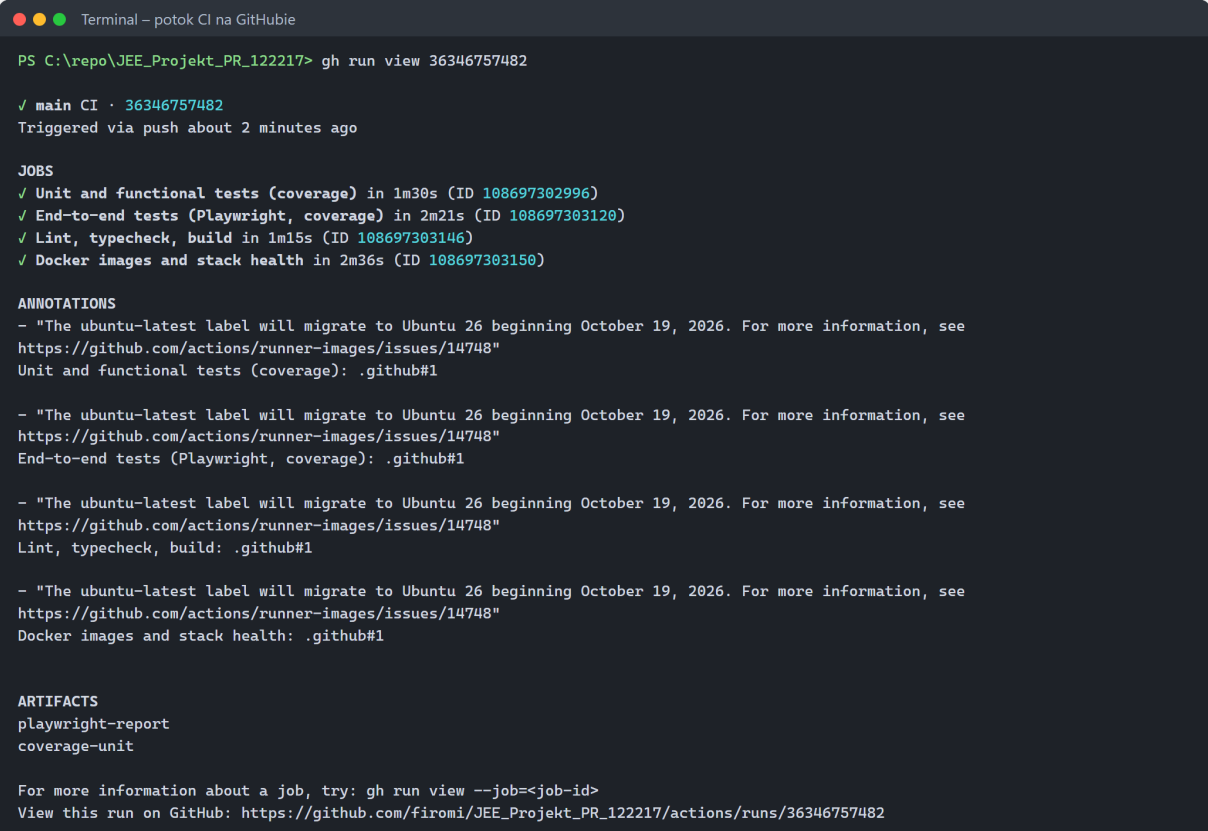
Rysunek 13: Testy E2E w przeglądarce (Playwright, emulacja telefonu Pixel 7)

Źródło: opracowanie własne (zrzut terminala)

5.5.10 Ciągła integracja

Plik `.github/workflows/ci.yml` opisuje potok GitHub Actions uruchamiany przy każdym wysłaniu zmian do repozytorium. Składa się z czterech zadań: kontroli kodu (lint, typy, budowanie, konfiguracja monitoringu), testów jednostkowych i funkcjonalnych z pokryciem, testów Playwright z pokryciem oraz zbudowania i uruchomienia całego stosu Docker z oczekiwaniem na stan *healthy*. Raporty trafiają do artefaktów. Poprawność pliku sprawdzono narzędziem `actionlint`.

Pierwsze przebiegi potoku na GitHubie wykryły trzy problemy, których nie widziałem na swoim komputerze, bo obrazy Docker i zbudowane pakiety były już na dysku: obrazy MinIO z rejestru `quay.io` wymagały logowania (rozdział 5.4), Playwright uruchamiał backend przed zbudowaniem pakietu `shared` i migracją bazy (przygotowanie przeniesiono do polecenia startowego serwera), a opisany wyżej wyścig filtrów pojawiał się pod większym obciążeniem. Po poprawkach wszystkie cztery zadania kończą się powodzeniem (rysunek 14).



```
Terminal - potok CI na GitHubie

PS C:\repo\JEE_Projekt_PR_122217> gh run view 36346757482

✓ main CI · 36346757482
Triggered via push about 2 minutes ago

JOBS
✓ Unit and functional tests (coverage) in 1m30s (ID 108697302996)
✓ End-to-end tests (Playwright, coverage) in 2m21s (ID 108697303120)
✓ Lint, typecheck, build in 1m15s (ID 108697303146)
✓ Docker images and stack health in 2m36s (ID 108697303150)

ANNOTATIONS
- "The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see
https://github.com/actions/runner-images/issues/14748"
Unit and functional tests (coverage): .github#1

- "The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see
https://github.com/actions/runner-images/issues/14748"
End-to-end tests (Playwright, coverage): .github#1

- "The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see
https://github.com/actions/runner-images/issues/14748"
Lint, typecheck, build: .github#1

- "The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see
https://github.com/actions/runner-images/issues/14748"
Docker images and stack health: .github#1

ARTIFACTS
playwright-report
coverage-unit

For more information about a job, try: gh run view --job=<job-id>
View this run on GitHub: https://github.com/firomi/JEE_Projekt_PR_122217/actions/runs/36346757482
```

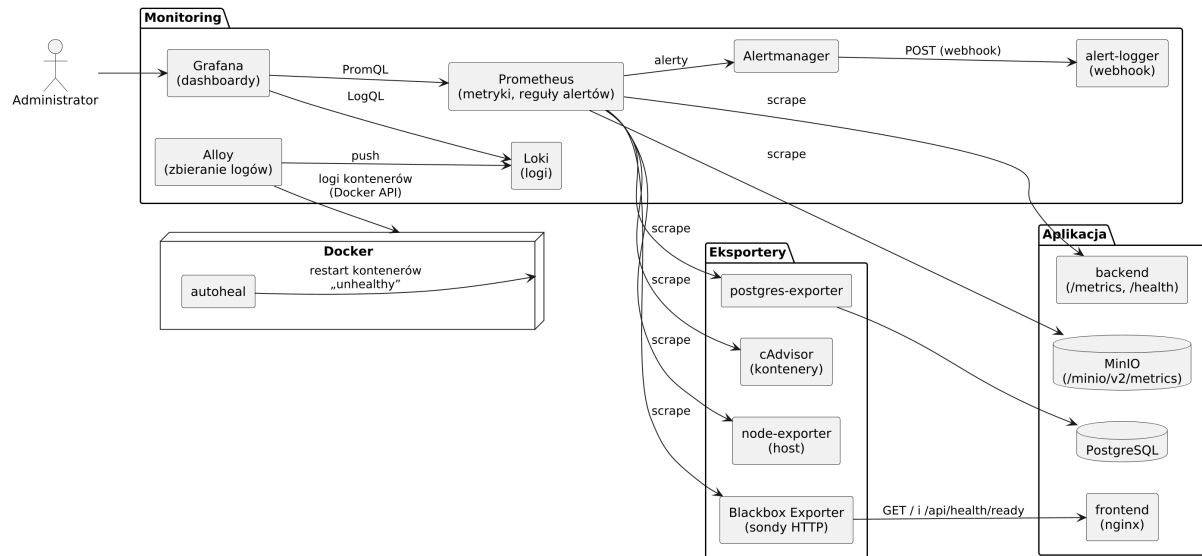
Rysunek 14: Przebieg potoku CI w serwisie GitHub (`gh run view`)

Źródło: opracowanie własne (zrzut terminala)

5.6 Monitoring

Nadzór nad systemem działa na czterech poziomach: procesu, kontenera, usługi widzianej z zewnątrz oraz alertów. Każda awaria ma być wykryta, w miarę możliwości naprawiona au-

tomatycznie i zgłoszona. Uzupełnieniem są metryki, dashboardy i centralne logi. Architekturę przedstawia rysunek 15.



Rysunek 15: Architektura monitoringu i nadzoru

Źródło: opracowanie własne

5.6.1 Metryki

Backend udostępnia endpoint `/metrics` w formacie tekstowym Prometheusa (biblioteka `prom-client`). Poza standardowymi metrykami procesu Node.js (zużycie procesora i pamięci, odświeżanie, opóźnienie pętli zdarzeń) zawiera metryki aplikacji:

- `http_requests_total` i `http_request_duration_seconds` – liczba i czas żądań według metody, kodu odpowiedzi i szablonu trasy (np. `/api/documents/:id`). Konkretnie adresy nie są etykietami, bo każdy identyfikator dokumentu tworzyłby nowy szereg czasowy. Żądania, które nie pasują do żadnej trasy, mają etykietę `unmatched`;
- `document_uploads_total` i `document_upload_size_bytes` – liczba przesłanych plików (według typu dokumentu) i ich rozmiar;
- `auth_login_failures_total` – odrzucone logowania, podstawa alertu o próbie zgadywania haseł;
- `db_query_duration_seconds` – czas zapytań do bazy, mierzony na podstawie zdarzeń zapytań klienta Prisma.

Endpoint `/metrics` leży poza ścieżką `/api`, dlatego serwer nginx go nie udostępnia. Prometheus pobiera go przez wewnętrzną sieć Dockera. Dodatkowe dane zbierają eksportery: `postgres-exporter` (połączenia, blokady, rozmiar bazy, transakcje), `cAdvisor` (procesor, pamięć i restarty każdego kontenera), `node-exporter` (zasoby komputera: procesor, pamięć, dysk) oraz wbudowany endpoint metryk MinIO (zajętość i liczba obiektów).

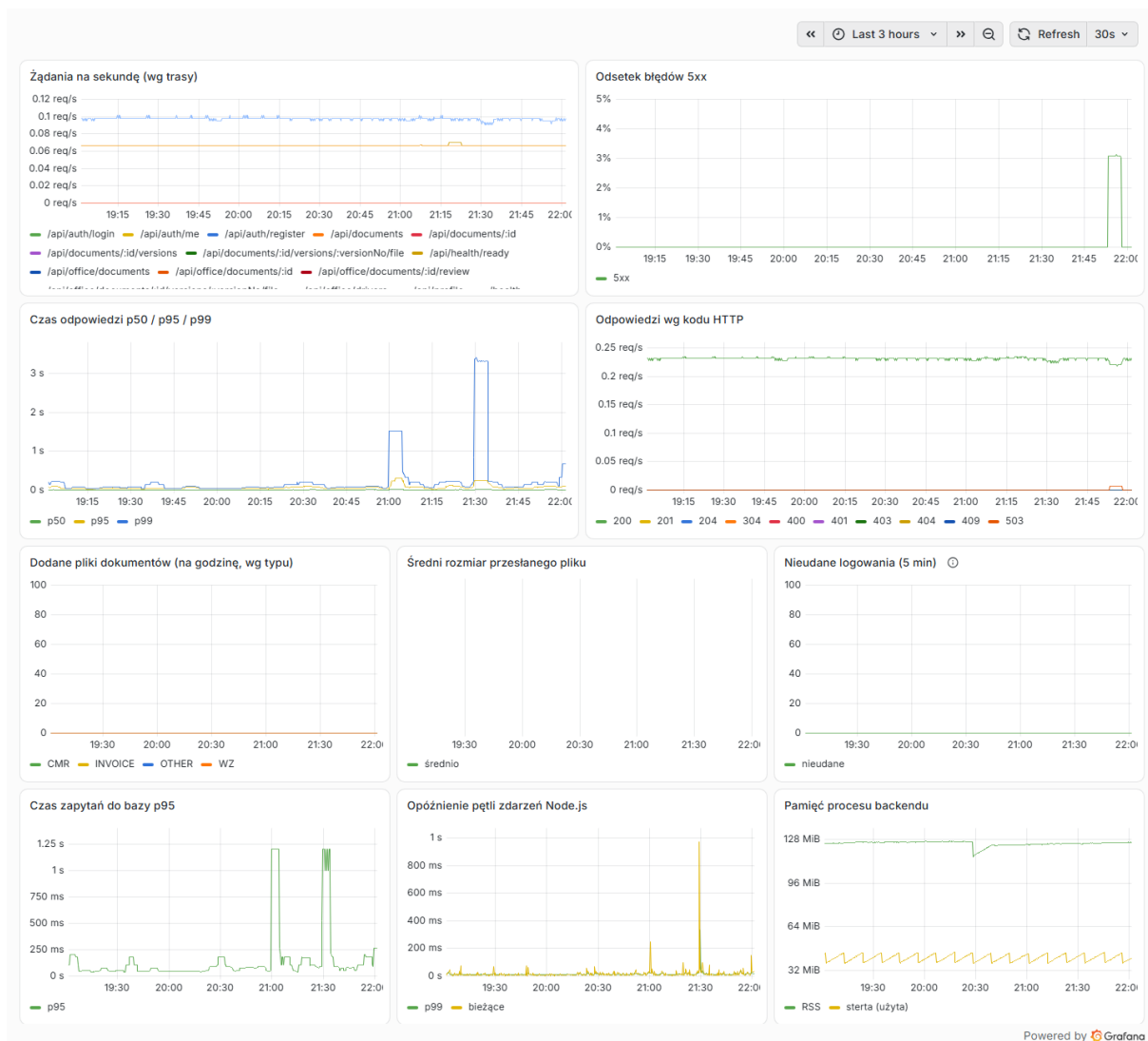
5.6.2 Dashboardy

Grafana przy starcie automatycznie dodaje źródła danych (Prometheus, Loki, Alertmanager) i cztery dashboardy przechowywane w repozytorium jako pliki JSON (`monitoring/grafana/dashboards`): „Aplikacja – API” (żądania na sekundę, odsetek błędów 5xx, czasy odpowiedzi p50/p95/p99, przesłane pliki, nieudane logowania, czas zapytań, opóźnienie pętli zdarzeń, pamięć), „Dostępność” (stan sond, dostępność w procentach z 24 godzin, czas odpowiedzi), „Kontenery i host” oraz „Baza danych i magazyn plików”.

Rysunki 16 i 17 pokazują dashboardy działającego systemu z ostatnich trzech godzin. Zrzuty wykonuje skrypt `scripts/grafana-screenshots.mjs`, który loguje się do Grafany w przeglądarce Chromium i czeka, aż wszystkie panele pobiorą dane. Ruch widoczny na wykresach pochodzi głównie z sond blackbox i kontroli stanu. Około godziny 21:55 wykresy odsetka błędów, czasu p99 i opóźnienia pętli zdarzeń pokazują krótki skok: na tym samym komputerze działały wtedy równolegle testy E2E i testy backendu, pętla zdarzeń backendu była zablokowana przez 2,4 s, a kontrola żywotności odpowiadała kodem 503. Monitoring zarejestrował rzeczywiste przeciążenie, a po jego ustąpieniu wartości wróciły do normy.

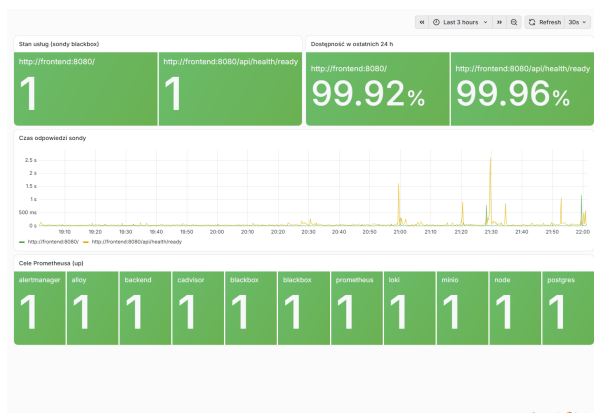
5.6.3 Watchdog procesu

- **Zasada fail fast.** Nieobsłużony wyjątek lub odrzucona obietnica powoduje zapis do logu i zakończenie procesu z kodem 1. Proces nie działa dalej w nieznanym stanie. Docker natychmiast uruchamia go od nowa (`restart: unless-stopped`).
- **Łagodne zamykanie.** Po sygnale SIGTERM serwer przestaje przyjmować połączenia, kończy obsługę bieżących żądań, zamyka pulę połączeń z bazą i klienta MinIO. Twardy limit wynosi 10 s.
- **Limity czasu.** Klient ma 20 s na przesłanie nagłówków i 60 s na całe żądanie (ochrona przed bardzo wolnymi klientami, a jednocześnie wystarczająco dużo na plik 10 MB z telefonu). PostgreSQL przerywa zapytanie po 10 s (`statement_timeout`), sterownik rezygnuje po 15 s, a połączenie z bazą i z MinIO musi powstać w ciągu kilku sekund. Zawieszona zależność nie zawiesza przez to całego API.
- **Zablokowana pętla zdarzeń.** Node.js obsługuje wszystkie żądania w jednym wątku, dlatego zapętłony synchroniczny kod blokuje całą aplikację, choć proces formalnie działa. Zegar uruchamiany co 100 ms mierzy, o ile później się wykonał, niż powinien, i zapamiętuje największe opóźnienie. Jeśli od poprzedniego sprawdzenia przekroczyło ono próg (domyślnie 1 s), GET `/health` odpowiada 503. Wbudowana funkcja `monitorEventLoopDelay` nie nadawała się do tego celu: test wykazał, że nie rejestruje blokady występującej bezpośrednio po wyzerowaniu histogramu.

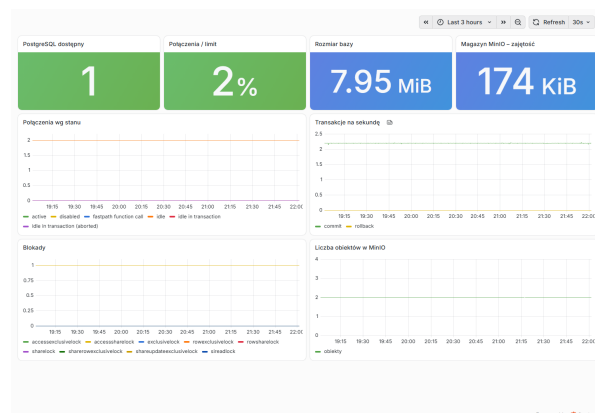


Rysunek 16: Dashboard „Aplikacja – API” w Grafanie

Źródło: opracowanie własne (zrzut ekranu)



(a) Dostępność



(b) Baza danych i magazyn plików

Rysunek 17: Dashboardsi dostępności oraz bazy danych i magazynu plików

Źródło: opracowanie własne (zrzut ekranu)

5.6.4 Watchdog kontenerów

Każda usługa ma politykę `restart`: `unless-stopped`, a wszystkie usługi, których obraz na to pozwala, mają `healthcheck`. Docker Compose restartuje jednak kontener tylko wtedy, gdy jego proces się zakończy. Kontener, który działa, ale jest oznaczony jako *unhealthy* (np. proces zawieszony), nie jest restartowany. Dlatego dodano kontener **autoheal**: co 5 sekund sprawdza on kontenery z etykietą `autoheal=true` i restartuje te, które stały się *unhealthy*. Gniazdo Dockera jest mu udostępnione tylko do odczytu. W Kubernetesie tę samą rolę pełni wbudowana *livenessProbe* i dodatkowy kontener nie byłby potrzebny. Obrazy Loki i Alloy nie mają powłoki (obrazy *distroless*), przez co nie da się w nich wykonać polecenia `healthcheck`. Ich stan kontroluje Prometheus, który pobiera z nich metryki (`alert TargetDown`).

Każda usługa ma limit pamięci (`mem_limit`) i procesora (`cpus`). Wyciek pamięci kończy się kontrolowanym restartem kontenera, a nie spowolnieniem całego komputera. Limity trzeba jednak dobrać do rzeczywistego zużycia. Grafana miała początkowo 256 MiB i podczas wykonywania zrzutów dashboardów została zatrzymana przez mechanizm OOM (zdarzenie `oom` w Dockerze, kod wyjścia 137). Po wczytaniu dashboardów trzyma ona około 430 MB obiektów, a domyślne ustawienia odświeżania pamięci w języku Go uruchamiają je dopiero przy podwojeniu sterty. Limit zwiększono do 768 MiB, a zmienna `GOMEMLIMIT` (600 MiB) wymusza wcześniejsze odświeżanie.

5.6.5 Sondy z zewnątrz

Blackbox Exporter co 15 sekund wysyła żądania HTTP tak jak użytkownik: przez serwer `nginx` do strony głównej i do `/api/health/ready`. Wykrywa to awarie niewidoczne w metrykach od środka, np. działający backend, do którego `nginx` nie przekazuje żądań. Wynik (`probe_success`) i czas odpowiedzi trafiają do Prometheusa i na dashboard dostępności.

5.6.6 Alerty

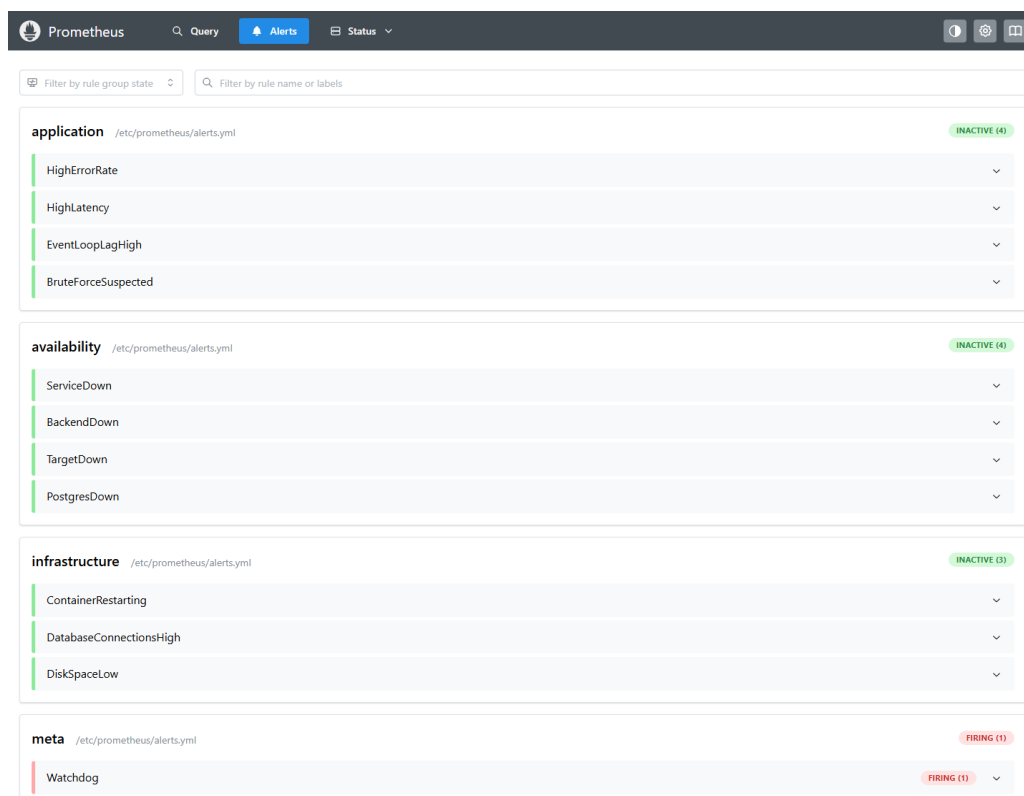
Reguły alertów są zapisane w repozytorium (`monitoring/prometheus/alerts.yml`) i sprawdzane narzędziem `promtool`. Tabela 13 zawiera ich listę. Alertmanager grupuje alerty i wysyła powiadomienia jako żądania HTTP (*webhook*) do kontenera `alert-logger`, który zapisuje je w swoim logu, a stamtąd trafiają do Loki. W rzeczywistym wdrożeniu wystarczy dodać odbiorcę e-mail, Discord lub Slack w pliku konfiguracyjnym Alertmanagera. Reguła wyciszania pomija alerty o błędach i opóźnieniach, gdy usługa jest całkowicie niedostępna.

Alert Watchdog jest aktywny przez cały czas i co 5 minut trafia do odbiorcy. Jego brak oznacza awarię samego monitoringu (Prometheusa, Alertmanagera albo połączenia między nimi), której zwykle alerty nie mogłyby zgłosić. To samo rozwiązanie stosuje standardowy zestaw `kube-prometheus`. Rysunek 18 pokazuje listę reguł w interfejsie Prometheusa. W chwili wykonania zrzutu żaden alert nie był aktywny.

Tabela 13: Reguły alertów

Alert	Warunek
Watchdog	Zawsze aktywny (<i>dead man's switch</i>)
ServiceDown	Sonda blackbox bez odpowiedzi 2xx przez 1 min
BackendDown, TargetDown	Prometheus nie może pobrać metryk (1 min / 2 min)
PostgresDown	Baza nie odpowiada eksporterowi przez 1 min
HighErrorRate	Ponad 5% odpowiedzi 5xx przez 5 min
HighLatency	Czas odpowiedzi p95 powyżej 1 s przez 5 min
EventLoopLagHigh	Opóźnienie pętli zdarzeń powyżej 0,5 s przez 2 min
BruteForceSuspected	Ponad 20 nieudanych logowań w 5 min
ContainerRestarting	Ponad 3 restarty kontenera w 15 min
DatabaseConnectionsHigh	Ponad 80% limitu połączeń PostgreSQL przez 5 min
DiskSpaceLow	Mniej niż 10% wolnego miejsca przez 10 min

Źródło: opracowanie własne



Rysunek 18: Reguły alertów w Prometheusie (wszystkie nieaktywne)

Źródło: opracowanie własne (zrzut ekranu)

5.6.7 Logi

Grafana Alloy odczytuje logi wszystkich kontenerów projektu przez API Dockera i wysyła je do Loki z etykietami `service` i `container`. Pierwotny plan zakładał narzędzie Promtail, ale Grafana wycofała je na rzecz Alloy. Logi backendu są w formacie JSON, dzięki czemu w Grafanie (widok *Explore*) można je filtrować po polach, np. `{service="backend"}` `json | requestId="..."`, i powiązać log żądania z błędem, który wystąpił podczas jego obsługi.

5.6.8 Test watchdogów (chaos test)

Skrypt `monitoring/chaos-test.sh` (`npm run chaos`) celowo psuje działający system i sprawdza, czy awaria zostaje wykryta i naprawiona albo zgłoszona. Wyniki przebiegu na komputerze deweloperskim (Docker Desktop, Windows) zawiera tabela 14.

Tabela 14: Wyniki testu watchdogów

Awaria	Reakcja systemu
Awaria procesu backendu (<code>kill -KILL</code> w kontenerze)	Polityka restartu uruchomiła kontener po 3 s, stan <i>healthy</i> po kolejnych 6 s
Zatrzymanie backendu (<code>docker stop</code> , <code>SIGTERM</code>)	Łagodne zamknięcie w 2 s, kod wyjścia 0
Zamrożenie procesu (<code>kill -STOP</code>)	Po 44 s (3 nieudane healthchecki) Docker oznaczył kontener jako <i>unhealthy</i> , autoheal zrestartował go po 14 s, po kolejnych 14 s kontener był <i>healthy</i>
Zatrzymanie PostgreSQL	Alert <code>ServiceDown</code> po 87 s (sonda co 15 s, warunek przez 1 min), <code>PostgresDown</code> w tym samym czasie; po uruchomieniu bazy alert wygasł w ciągu 20 s
Konfiguracja monitoringu	<code>promtool</code> : konfiguracja i 12 reguł poprawne; <code>amtool</code> : konfiguracja poprawna

Źródło: opracowanie własne

Wnioski z testu. Plan zakładał symulowanie awarii poleceniem `docker kill`. Pierwszy przebieg pokazał, że po nim kontener *nie* wraca: Docker traktuje `docker kill` jak ręczne zatrzymanie, do którego polityka restartu się nie stosuje. Dotyczy ona tylko sytuacji, gdy proces zakończy się sam. Prawdziwą awarię (np. przerwanie procesu przez system z powodu braku pamięci) symuluje dopiero zabicie procesu Node.js wewnątrz kontenera. Ten sam przebieg ujawnił też dwa błędy samego skryptu: fałszywy wynik „OK” przy odczycie danych zatrzymanego kontenera i alert pozostały po poprzednim scenariuszu, który zaburzał pomiar kolejnego. Poprawiony skrypt czeka na czysty stan przed każdym scenariuszem.

6 Podsumowanie i wnioski

6.1 Realizacja celu

Zrealizowano wszystkie wymagania funkcjonalne z rozdziału 2. Kierowca zakłada konto i loguje się, a żądania są uwierzytelniane tokenem JWT. Uzupełnia profil, do którego przypisane są jego dokumenty. Dokument fotografuje aparatem telefonu, kadruje i obraca zdjęcie, a aplikacja kompresuje je przed wysłaniem (zdjęcie 12 Mpx o rozmiarze 3,7 MB zajmuje po kompresji około 0,7–0,8 MB). Dokumenty mają typy, statusy z regułami przejść, kolejne wersje pliku i historię zmian. Można je wyszukiwać i filtrować. Przesłany dokument ocenia biuro w panelu w przeglądarce na komputerze: akceptuje go albo odrzuca z podaniem powodu, który kierowca widzi na telefonie. Zdjęcie dokumentu, jego wysłanie, instalację PWA oraz odrzucenie i ponowne przesłanie sprawdzono na prawdziwym telefonie (podrozdział 5.5).

Wymagania нефункционалне zrealizowano następująco:

- **Bezpieczeństwo:** pliki są szyfrowane algorytmem AES-256-GCM z osobnym kluczem dla każdego pliku (szyfrowanie kopertowe), hasła są przechowywane jako skróty argon2id, a nieudane logowania ograniczane. Użytkownik ma dostęp tylko do własnych dokumentów, co sprawdzają testy.
- **Obsługa mobilna:** responsywny interfejs z dolną nawigacją, instalowalny jako PWA i uruchamiający się bez sieci.
- **Łatwe wdrożenie:** cały system, łącznie z monitoringiem, uruchamia polecenie `docker compose up`. Migracje bazy wykonuje osobny kontener przed startem backendu.
- **Obserwowalność i niezawodność:** metryki Prometheus, cztery dashboardy Grafany, 12 reguł alertów, centralne logi w Loki oraz automatyczne restarty po awarii i po zawieszeniu procesu, sprawdzone testem chaosu.
- **Jakość:** 189 testów backendu (głównie funkcjonalnych, na prawdziwej bazie i magazynie plików), 109 testów pakietu wspólnego, 62 testy jednostkowe frontendu i 36 scenariuszy Playwright w emulacji telefonu i przeglądarki na komputerze. Pokrycie kodu przekracza 93 % instrukcji w każdym pakiecie (frontend mierzony testami w przeglądarce).

6.2 Napotkane problemy i wnioski

Testy funkcjonalne wykryły kilka błędów, których nie ujawniłyby testy jednostkowe ani pobieżne ręczne sprawdzenie:

- po odświeżeniu strony pierwsze zapytania wysyłane były bez tokenu, bo efekty komponentów potomnych w React wykonują się przed efektem dostawcy kontekstu (poprawka: token przechowywany poza drzewem komponentów);

- strumień aparatu wyłączał się zaraz po uruchomieniu, a użytkownik widziałby zamrożoną klatkę (poprawka: efekt uruchamiany raz przy wyświetleniu widoku);
- po wylogowaniu i zalogowaniu innego kierowcy aplikacja wracała na stronę dokumentu poprzedniego użytkownika;
- metryki HTTP miały błędne etykiety tras, bo Express przywraca ścieżkę bazową routera przed zakończeniem odpowiedzi;
- pomiar opóźnienia pętli zdarzeń tracił próbkę po wyzerowaniu histogramu i zawieszenie procesu mogło pozostać niewykryte;
- po odrzuceniu dokumentu przez biuro i ponownym przesłaniu go przez kierowcę biuro nadal widziało stary status, bo pamięć podręczna uznawała dane za aktualne przez 30 sekund;
- filtr statusu wybrany w ciągu 300 ms od wpisania tekstu w wyszukiwarce był cofany przez opóźnione wyszukiwanie.

Do każdego z tych błędów napisano test odtwarzający go przed poprawką kodu. Testy pozostają w zestawie i chronią przed powrotem błędu.

Testy w przeglądarce okazały się wrażliwe na obciążenie komputera: haszowanie haseł `argon2` i przetwarzanie zdjęć 12 Mpx konkurowały o procesor z przeglądarkami. Stabilność zapewniło uruchamianie testów na zbudowanej aplikacji (zamiast serwera deweloperskiego) i wykonywanie scenariuszy po kolei.

Sama konfiguracja kontroli stanu nie gwarantuje samonaprawy. Test chaosu pokazał, że Docker nie restartuje kontenera zatrzymanego poleceniem `docker kill`, bo traktuje je jak ręczne zatrzymanie, a zawieszzonego, ale działającego procesu nie restartuje wcale. Dopiero dodatkowy kontener *autoheal* i test z rzeczywistym zawieszeniem procesu potwierdziły, że system sam wraca do działania.

6.3 Ograniczenia

Na prawdziwym urządzeniu aplikację sprawdzono na jednym telefonie z systemem Android. Inne urządzenia, w tym iPhone z przeglądarką Safari, nie były testowane. Konto biura jest jedno i powstaje z konfiguracji serwera. Aplikacja nie ma zarządzania pracownikami biura (dodawania kont, ról, podziału kierowców między dyspozytorów).

6.4 Możliwości rozwoju

- **Lepsze zdjęcia dokumentów:** automatyczne wykrywanie krawędzi kartki i korekcja perspektywy, jak w aplikacjach skanujących (podrozdział 3).

- **Rozpoznawanie tekstu (OCR):** odczyt numeru dokumentu, nadawcy i odbiorcy ze zdjęcia i wypełnianie nimi formularza.
- **Rozbudowa panelu biura:** zarządzanie kontami pracowników biura, przypisanie kierowców do dyspozytorów i powiadomienia (push, e-mail) o decyzji biura oraz o nowych dokumentach do sprawdzenia.
- **e-CMR i eFTI:** import i eksport elektronicznych listów przewozowych oraz integracja z platformami eFTI, które od 2027 r. mają być w pełni honorowane przez organy kontrolne.
- **Praca bez sieci:** kolejka wysyłek zapisywana na urządzeniu w postaci zaszyfrowanej i wysyłana po odzyskaniu połączenia, co ma znaczenie na trasach bez zasięgu.
- **Zarządzanie kluczami:** przechowywanie klucza głównego w usłudze KMS i jego okresowa rotacja (szyfrowanie kopertowe pozwala zmienić klucz główny bez ponownego szyfrowania plików).

Spis tabel

1	Porównanie istniejących rozwiązań z Aplikacją kierowcy	7
2	Technologie użyte w projekcie	9
3	Katalogi repozytorium	10
4	Kompresja zdjęć przed wysłaniem (pomiar w przeglądarce Chromium)	15
5	Kody błędów API	21
6	Endpointy API	22
7	Tabela <code>users</code>	30
8	Tabela <code>driver_profiles</code>	30
9	Usługi zdefiniowane w pliku <code>docker-compose.yml</code>	32
10	Zakres testów	36
11	Pozostałe błędy wykryte przez testy	37
12	Pokrycie kodu testami	40
13	Reguły alertów	48
14	Wyniki testu watchdogów	49

Spis rysunków

1	Diagram kontenerów systemu (model C4)	8
2	Przepływ zdjęcia: aparat, kadrowanie, kompresja, wysłanie	14
3	Panel biura w przeglądarce na komputerze	18
4	Ekrany kierowcy na telefonie: logowanie, lista, szczegóły dokumentu i profil . .	19
5	Dodawanie dokumentu zdjęciem na telefonie	19
6	Sekwencja logowania i użycia tokenu JWT	24
7	Sekwencja dodania dokumentu z szyfrowaniem kopertowym	26
8	Diagram stanów dokumentu	28
9	Diagram ERD bazy danych	31
10	Test na prawdziwym telefonie (Samsung Galaxy A26, Android 16, Chrome) . .	39
11	Testy backendu (funkcjonalne API i jednostkowe)	40
12	Testy jednostkowe pakietu <code>shared</code> i frontendu	41
13	Testy E2E w przeglądarce (Playwright, emulacja telefonu Pixel 7)	42
14	Przebieg potoku CI w serwisie GitHub (<code>gh run view</code>)	43
15	Architektura monitoringu i nadzoru	44
16	Dashboard „Aplikacja – API” w Grafanie	46
17	Dashboardsy dostępności oraz bazy danych i magazynu plików	46
18	Reguły alertów w Prometheusie (wszystkie nieaktywne)	48

Bibliografia

- [1] *Konwencja o umowie międzynarodowego przewozu drogowego towarów (CMR), sporządzona w Genewie dnia 19 maja 1956 r.* Dz.U. 1962 nr 49 poz. 238. 1956.
- [2] *Additional Protocol to the Convention on the Contract for the International Carriage of Goods by Road (CMR) concerning the Electronic Consignment Note, Geneva, 20 February 2008 – status.* United Nations Treaty Collection. URL: https://treaties.un.org/Pages/ViewDetails.aspx?src=TREATY&mtdsg_no=XI-B-11-b&chapter=11&clang=_en (dostęp 24.09.2026).
- [3] *Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2020/1056 z dnia 15 lipca 2020 r. w sprawie elektronicznych informacji dotyczących przewozu towarów.* Dz.U. UE L 249 z 31.07.2020. 2020. URL: <https://eur-lex.europa.eu/eli/reg/2020/1056/oj> (dostęp 24.09.2026).
- [4] *TransFollow – Digital Transport Solutions & eCMR.* TransFollow. URL: <https://transfollow.com/> (dostęp 24.09.2026).
- [5] *Regulations of the Trans.eu Platform (effective 01.09.2025).* Trans.eu Group S.A. URL: https://www.trans.eu/wp-content/uploads/2025/07/ES_Regulations-of-the-TRANS.EU-PLATFORM-01.09.2025.pdf (dostęp 24.09.2026).
- [6] *eCMR: The Digital Future of Transport Documentation.* CargoON (Trans.eu Group S.A.) 2025. URL: <https://resources.cargoone.eu/en/community/blog/digitalisation/ecmr-digital-future-of-transport-documentation/> (dostęp 24.09.2026).
- [7] *Adobe Scan: free mobile PDF scanner app.* Adobe. URL: <https://www.adobe.com/acrobat/mobile/scanner-app.html> (dostęp 24.09.2026).
- [8] *MediaDevices: getUserMedia() method.* MDN Web Docs, Mozilla. URL: <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia> (dostęp 23.09.2026).
- [9] *Password Storage Cheat Sheet.* OWASP Foundation. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (dostęp 23.09.2026).
- [10] M. B. Jones, J. Bradley i N. Sakimura. *JSON Web Token (JWT).* RFC 7519, Internet Engineering Task Force. 2015. URL: <https://www.rfc-editor.org/rfc/rfc7519>.
- [11] M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC.* NIST Special Publication 800-38D, National Institute of Standards and Technology. 2007. URL: <https://doi.org/10.6028/NIST.SP.800-38D>.